



**Politechnika  
Śląska**

## **PROJEKT INŻYNIERSKI**

System wspomagający inteligentne składanie zestawów komputerowych

**Piotr KUPCZYK**

Nr albumu: 305198

**Kierunek:** Teleinformatyka S1

**Specjalność:** Brak

**PROWADZĄCY PRACĘ**

**dr hab. inż. Bożena Małysiak-Mrozek**

**KATEDRA Systemów Rozproszonych i Urządzeń Informatyki**

**Wydział Automatyki, Elektroniki i Informatyki**

**Gliwice 2025**



**Tytuł pracy**

System wspomagający inteligentne składanie zestawów komputerowych

**Streszczenie**

(Streszczenie pracy – odpowiednie pole w systemie APD powinno zawierać kopię tego streszczenia.)

**Słowa kluczowe**

(2-5 słów (fraz) kluczowych, oddzielonych przecinkami)

**Thesis title**

A system supporting intelligent assembly of computer sets

**Abstract**

(Thesis abstract – to be copied into an appropriate field during an electronic submission – in English.)

**Key words**

(2-5 keywords, separated by commas)



# Spis treści

|          |                                                                                             |           |
|----------|---------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Wstęp</b>                                                                                | <b>1</b>  |
| 1.1      | Wprowadzenie do problemu . . . . .                                                          | 1         |
| 1.2      | Osadzenie problemu w dziedzinie . . . . .                                                   | 1         |
| 1.3      | Cel pracy . . . . .                                                                         | 1         |
| 1.4      | Zakres prac . . . . .                                                                       | 2         |
| 1.5      | Zwięzła charakterystyka rozdziałów . . . . .                                                | 3         |
| 1.6      | Wkład autora . . . . .                                                                      | 4         |
| <b>2</b> | <b>Analiza problemu oraz przegląd dostępnych rozwiązań</b>                                  | <b>5</b>  |
| 2.1      | Sformułowanie problemu . . . . .                                                            | 5         |
| 2.2      | Analiza istniejących rozwiązań w najpopularniejszych sklepach na polskim<br>rynku . . . . . | 6         |
| 2.3      | Porównanie istniejących rozwiązań z rozwiązaniem inżynierskim . . . . .                     | 6         |
| <b>3</b> | <b>Wymagania i narzędzia</b>                                                                | <b>9</b>  |
| 3.1      | Wymagania funkcjonalne . . . . .                                                            | 9         |
| 3.2      | Wymagania нефункционалне . . . . .                                                          | 11        |
| 3.3      | Przypadki użycia . . . . .                                                                  | 11        |
| 3.4      | Narzędzia i technologie . . . . .                                                           | 12        |
| 3.4.1    | Frontend . . . . .                                                                          | 12        |
| 3.4.2    | Backend . . . . .                                                                           | 12        |
| 3.4.3    | Modelowanie i narzędzia wspomagające . . . . .                                              | 13        |
| 3.4.4    | Modelowanie i narzędzia wspomagające . . . . .                                              | 13        |
| 3.5      | Metodyka pracy . . . . .                                                                    | 13        |
| <b>4</b> | <b>Specyfikacja zewnętrzna</b>                                                              | <b>15</b> |
| 4.1      | Sposób instalacji systemu . . . . .                                                         | 15        |
| 4.2      | Kategorie użytkowników . . . . .                                                            | 16        |
| 4.3      | Sposób obsługi systemu . . . . .                                                            | 17        |
| 4.4      | Administracja systemem . . . . .                                                            | 17        |
| 4.5      | Kwestie bezpieczeństwa . . . . .                                                            | 18        |

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| 4.6      | Scenariusze korzystania z systemu . . . . .                             | 18        |
| 4.6.1    | Scenariusz 1: Przeglądanie katalogu komponentów . . . . .               | 19        |
| 4.6.2    | Scenariusz 2: Filtrowanie oraz wybór produktów . . . . .                | 19        |
| 4.6.3    | Scenariusz 3: Dodawanie produktów do koszyka . . . . .                  | 20        |
| 4.6.4    | Scenariusz 4: Poprawne logowanie użytkownika . . . . .                  | 21        |
| 4.6.5    | Scenariusz 5: Niepoprawne logowanie użytkownika . . . . .               | 22        |
| 4.6.6    | Scenariusz 6: Utworzenie kompatybilnego zestawu komputerowego . . . . . | 22        |
| 4.6.7    | Scenariusz 7: Próba utworzenia niekompatybilnego zestawu . . . . .      | 24        |
| <b>5</b> | <b>Specyfikacja wewnętrzna</b>                                          | <b>25</b> |
| 5.1      | Przedstawienie idei systemu . . . . .                                   | 25        |
| 5.1.1    | Cel powstania aplikacji . . . . .                                       | 25        |
| 5.1.2    | Innowacyjność systemu . . . . .                                         | 26        |
| 5.2      | Architektura systemu . . . . .                                          | 26        |
| 5.2.1    | Schemat ideowy . . . . .                                                | 27        |
| 5.2.2    | Frontend . . . . .                                                      | 29        |
| 5.2.3    | Backend . . . . .                                                       | 30        |
| 5.2.4    | Baza danych . . . . .                                                   | 31        |
| 5.2.5    | Infrastruktura chmurowa . . . . .                                       | 32        |
| 5.3      | Baza danych . . . . .                                                   | 36        |
| 5.3.1    | Model danych - ERD . . . . .                                            | 36        |
| 5.3.2    | Opis najważniejszych encji . . . . .                                    | 39        |
| 5.4      | Komponenty i moduły systemu . . . . .                                   | 52        |
| 5.4.1    | Moduł kompatybilności podzespołów . . . . .                             | 53        |
| 5.4.2    | Moduł logiki rozmytej . . . . .                                         | 53        |
| 5.4.3    | Moduł zarządzania koszykiem i zamówieniami . . . . .                    | 55        |
| 5.5      | Najważniejsze algorytmy . . . . .                                       | 56        |
| <b>6</b> | <b>Weryfikacja i walidacja</b>                                          | <b>57</b> |
| 6.1      | Cel weryfikacji systemu . . . . .                                       | 57        |
| 6.2      | Metody testowania . . . . .                                             | 57        |
| 6.3      | Organizacja eksperymentów . . . . .                                     | 58        |
| 6.4      | Przypadki testowe . . . . .                                             | 59        |
| 6.5      | Wyniki testów . . . . .                                                 | 60        |
| 6.6      | Wykryte i usunięte błędy . . . . .                                      | 60        |
| 6.7      | Ocena poprawności działania systemu . . . . .                           | 61        |
| <b>7</b> | <b>Podsumowanie i wnioski</b>                                           | <b>63</b> |
| 7.1      | Realizacja celów pracy . . . . .                                        | 63        |
| 7.2      | Ocena otrzymanych rezultatów . . . . .                                  | 63        |

|                               |                                                  |           |
|-------------------------------|--------------------------------------------------|-----------|
| 7.3                           | Trudności napotkane podczas realizacji . . . . . | 64        |
| 7.4                           | Możliwe kierunki dalszego rozwoju . . . . .      | 64        |
| 7.5                           | Wnioski końcowe . . . . .                        | 64        |
| <b>Bibliografia</b>           |                                                  | <b>67</b> |
| <b>Spis skrótów i symboli</b> |                                                  | <b>71</b> |
| <b>Spis rysunków</b>          |                                                  | <b>73</b> |
| <b>Spis tabel</b>             |                                                  | <b>75</b> |





# Rozdział 1

## Wstęp

Niniejszy rozdział stanowi wprowadzenie do tematyki pracy inżynierskiej. Przedstawiono w nim motywy realizacji tematu oraz zarys problemu, którego rozwiązanie stanowi przedmiot dalszych rozważań. W rozdziale określono również zakres pracy oraz jej główne cele.

### 1.1 Wprowadzenie do problemu

Sklepy komputerowe zazwyczaj posiadają swój własny sklep internetowy, oferujący możliwość zakupów części indywidualnie, jak i gotowych zestawów PC. Odpowiedni dobór komponentów jest najistotniejszą częścią procesu projektowania własnego stanowiska, a pomimo tego najwięksi sprzedawcy nie oferują możliwości sprawdzania kompatybilności wybranych produktów. Dlatego projekt ma na celu utworzenie prostego i intuicyjnego systemu internetowego, który zaoferuje przystępną formę weryfikacji poprawności wyboru komponentów z katalogu oraz finalnie dodanie kompatybilnego zestawu do koszyka użytkownika.

### 1.2 Osadzenie problemu w dziedzinie

Projekt osadzony jest w dziedzinie inżynierii oprogramowania, projektowania baz danych oraz systemów e-commerce. Praca obejmuje zagadnienia relacyjności baz danych, budowy warstwy komunikacyjnej backendu, implementacji logiki aplikacji pod kątem biznesowym oraz wdrożenie interfejsu frontendowego.

### 1.3 Cel pracy

Celem pracy jest zaprojektowanie i zaimplementowanie systemu webowego, który będzie pełnić funkcję inteligentnego kreatora zestawów komputerowych. System będzie da-

wać użytkownikowi możliwość wybrania dowolnych produktów z katalogu, sprawdzenie czy aktualne zamówienie jest wzajemnie kompatybilne, a gdy weryfikacja zakończy się pomyślnie, umożliwi realizację zamówienia.

Idące za tym cele to:

- projekt i utworzenie relacyjnej bazy danych, odpowiadającej potrzebom sklepu internetowego,
- zaprojektowanie i implementacja backendu, odpowiedzialnego za komunikację i logikę aplikacji,
- wdrożenie przejrzystego frontendu, będącego interfejsem klienta,
- utworzenie grupy zasobów w środowisku chmurowym Azure (baza danych jako storage, aplikacja jako kontener).

## 1.4 Zakres prac

Zakres prac nad projektem obejmuje:

- projekt relacyjnej bazy danych w Oracle SQL Developer.
- utworzenie środowiska testowego serwera PostgreSQL, wprowadzenie gotowych danych o produktach do bazy danych,
- utworzenie użytkowników oraz nadanie odpowiednich uprawnień każdemu z nich,
- implementację backendu w języku Python, implementacja API z wykorzystaniem frameworka FastAPI, umożliwiając integrację z bazą danych w celu zarządzania produktami i zamówieniami,
- projekt interfejsu użytkownika, stworzenie frontendu używając HTML, CSS oraz JavaScript,
- projekt kreatora zestawów komputerowych, zaimplementowanego z uwzględnieniem logik rozmytej, używając Reacta, Typescript, Node.js,
- migrację gotowej aplikacji z lokalnego serwera testowego PostgreSQL do produkcyjnej chmury Azure, utworzenie grupy zasobów.

## 1.5 Zwięzła charakterystyka rozdziałów

W podrozdziale zaprezentowana została struktura części pisemnej pracy.

- **Rozdział 1** - zawiera wprowadzenie do tematyki pracy, przedstawia problem nieintuicyjnego doboru kompatybilnych podzespołów komputerowych, definiuje cel i zakres pracy oraz opisuje autorski wkład w projekt.
- **Rozdział 2** - obejmuje analizę tematu, w tym istotę kompatybilności sprzętowej, przegląd istniejących rozwiązań wykorzystywanych przez popularne sklepy komputerowe oraz porównanie ich funkcjonalności z zaprojektowanym systemem. Przybliża problematykę całej pracy.
- **Rozdział 3** - przedstawia wymagania funkcjonalne i нефunkcjonalne systemu, przypadki użycia, zastosowane narzędzia i technologie oraz metodykę pracy przyjętą podczas implementacji. Przedstawia kompletną listę dobranych technologii i rozwiązań.
- **Rozdział 4** - stanowi część projektową, zawierającą architekturę systemu, opisanie każdego filaru aplikacji webowej, model danych wraz z diagramem ERD, opis struktury bazy danych, szczegółową analizę encji oraz omówienie kluczowych modułów, takich jak moduł weryfikacji kompatybilności i logiki rozmytej.
- **Rozdział 5** - prezentuje proces weryfikacji i walidacji systemu, zawierając opis sposobu testowania, organizacji eksperymentów, przypadków testowych oraz analizę wykrytych błędów i wyników testów. Analizuje skuteczność działania oprogramowania, porównując otrzymywane wyniki z założeniami.
- **Rozdział 6** - zawiera podsumowanie uzyskanych rezultatów, ocenę stopnia realizacji celów pracy oraz wskazuje możliwe kierunki dalszego rozwoju systemu. Podsumowuje i ocenia skuteczność podjętych działań, analizuje koszty poniesione podczas realizacji, szczegółowo ocenia dobór technologii.

## 1.6 Wkład autora

Autor samodzielnie opracował wszystkie elementy projektu, zaczynając od zaplanowania funkcjonalności oraz wyboru technologii. Następnym krokiem było zaprojektowanie relacyjnej bazy danych w Oracle SQL Developer, gdzie utworzył on testowy serwer PostgreSQL wraz z użytkownikami oraz nadał odpowiednie uprawnienia każdemu z nich. Przy użyciu frameworku FastAPI przygotowany został backend w języku Python, odpowiadający za komunikację z bazą danych oraz za logikę całej aplikacji, umożliwiając weryfikację kompatybilności zestawów komputerowych. Autor utworzył interaktywny frontend, który umożliwia graficzne korzystanie z aplikacji, zapewniając łatwość użytkowania zarówno dla doświadczonych, jak i początkujących inżynierów sprzętu, poprzez stosowanie logiki rozmytej. Korzystając z akademickiej subskrypcji Microsoft Azure autor używając otrzymanych kredytów samodzielnie realizuje hosting, administrację, testowanie CI/CD, konteneryzację oraz wirtualizację aplikacji webowej, czyniąc ją w pełni używalną w publicznej sieci internet. Dokumentacja przygotowana do projektu, dokument inżynierski oraz wszystkie opisy dostępne w projekcie zostały samodzielnie, szczegółowo opracowane w ramach projektu inżynierskiego "System wspomagający inteligentne składanie zestawów komputerowych".

## Rozdział 2

# Analiza problemu oraz przegląd dostępnych rozwiązań

Celem niniejszego rozdziału jest analiza problemu będącego przedmiotem pracy oraz przegląd istniejących rozwiązań stosowanych w podobnych systemach. Przeprowadzono analizę funkcjonalną dostępnych narzędzi oraz wskazano ich ograniczenia, które stanowiły podstawę do zaprojektowania własnego rozwiązania.

### 2.1 Sformułowanie problemu

#### Istota kompatybilności

Na rynku znajduje się szeroka gama komponentów komputerowych, która nieustannie poszerza się o nowe produkty. Różnice pomiędzy poszczególnymi modelami dotyczą chociażby wymiarów fizycznych, dostępnych złączy, technologii socketów, minimalnej mocy zasilacza, aż do wymogów software'owych. Odpowiedni dobór części jest pierwszym krokiem do realizacji poprawnie funkcjonującego stanowiska komputerowego, gdyż dowolna pomyłka może prowadzić do braku możliwości fizycznego montażu lub niewłaściwej i nieciągłej pracy urządzenia. Nawet minimalne niezgodności mogą być przyczyną poważnej awarii, która może przyczynić się do strat finansowych, spowodowanych utratą danych, przerwaniem trwałości usług lub całkowitym uszkodzeniem sprzętu.

#### Niska dostępność narzędzi do weryfikacji kompatybilności

Aktualnie na polskim rynku sklepów komputerowych brakuje narzędzi umożliwiających użytkownikowi samodzielną i automatyczną weryfikację kompatybilności wybranych podzespołów. Większość sprzedawców oferuje wyłącznie gotowe zestawy komputerowe, które zapewniają pełną funkcjonalność, jednak często nie umożliwiają ich łatwej rozbudowy ani wymiany poszczególnych elementów na nowsze modele. W efekcie zaprojektowanie stanowiska w pełni odpowiadającego potrzebom użytkownika staje się utrudnione

i może wymagać konsultacji ze specjalistą, co generuje dodatkowe koszty oraz wydłuża czas realizacji zamówienia.

## **2.2 Analiza istniejących rozwiązań w najpopularniejszych sklepach na polskim rynku**

Spośród wielu istniejących rozwiązań dostępnych na polskim rynku wybrano dwa, których poziom zaawansowania jest najwyższy i nie odbiega zbytnio od tego przyjętego przez autora.

### **Morele.net**

Spośród przeanalizowanych platform sprzedażowych polskich sprzedawców, pod względem technicznym najlepiej zaprojektowanym rozwiązaniem okazał się konfigurator zestawów PC dostępny w sklepie Morele[2]. Narzędzie oferuje intuicyjny interfejs oraz możliwość wyboru szerokiej gamy podzespołów. Kluczowym ograniczeniem jest jednak brak automatycznej weryfikacji kompatybilności, gdzie w przypadku błędnie skonfigurowanego zestawu jedynie pracownik sklepu może ewentualnie skontaktować się z użytkownikiem. W praktyce znacząco opóźnia to proces realizacji zamówienia i niesie ryzyko zakupu podzespołów, które nie będą ze sobą wzajemnie kompatybilne. Dodatkowo konfigurator nie wykorzystuje logiki rozmytej przy sugerowaniu komponentów.

### **X-kom.pl**

Drugim pod względem funkcjonalności rozwiązaniem jest konfigurator oraz oferta gotowych zestawów sklepu X-kom[4]. Platforma udostępnia czytelny interfejs oraz rekomendowane zestawy komputerowe, które zazwyczaj umożliwiają późniejszą rozbudowę. W porównaniu do Morele[2], użytkownik ma tutaj jeszcze mniejszą możliwość samodzielnego projektowania konfiguracji, ponieważ wybór ogranicza się do kilku lub kilkunastu proponowanych zestawów. Brak szczegółowej analizy potrzeb użytkownika oraz ograniczona liczba wariantów utrudniają złożenie w pełni dostosowanego zamówienia. Podobnie jak w przypadku Morele, sklep nie wykorzystuje logiki rozmytej w mechanizmach komponowania zestawów.

## **2.3 Porównanie istniejących rozwiązań z rozwiązaniem inżynierskim**

Pierwszą i najważniejszą funkcjonalnością wyróżniającą opracowane rozwiązanie jest zastosowanie rozszerzonych mechanizmów weryfikacji kompatybilności podzespołów, któ-

rych brakuje w analizowanych sklepach internetowych. System wykorzystuje nie tylko klasyczne metody porównywania parametrów technicznych, lecz również elementy logiki rozmytej pozwalającej na bardziej elastyczną ocenę dopasowania podzespołów do oczekiwań użytkownika.

Aplikacja dokonuje szczegółowej analizy technicznej wszystkich wybranych komponentów, co przekłada się na sprawdzanie i weryfikowanie:

- zgodności procesora i płyty głównej pod względem zastosowanego gniazda socket'u,
- typu pamięci RAM oraz jej zgodności ze standardem płyty głównej,
- maksymalnej obsługiwanej częstotliwości i pojemności modułów pamięci,
- parametrów energetycznych, takich jak zapotrzebowanie na moc (TDP) oraz wydajności i maksymalnej dostarczanej mocy zasilacza,
- wymiarów i formatów kart graficznych oraz obudowy,
- zgodności chłodzenia procesora z wybranym gniazdem i limitem wysokości w obudowie.

W odróżnieniu od popularnych serwisów takich jak Morele.net[2] czy X-kom[4], które nie oferują pełnej automatycznej weryfikacji kompatybilności, system analizuje wszystkie istotne zależności pomiędzy podzespołami. Dzięki temu użytkownik nie jest narażony na ryzyko zakupu części wzajemnie niekompatybilnych, a proces konfiguracji zestawu przebiega szybciej, sprawniej i bez konieczności konsultacji ze specjalistą. Zastosowanie logiki rozmytej pozwala ponadto na ocenę zestawów zarazem pod kątem poprawności technicznej, lecz także ich dopasowania do ogólnych kategorii takich jak budżet, wydajność czy energooszczędność.





# Rozdział 3

## Wymagania i narzędzia

W rozdziale tym określone zostały wymagania funkcjonalne i нефункционаłne projektowanego systemu oraz przedstawiono technologie i narzędzia wykorzystane podczas realizacji pracy. Zdefiniowane wymagania stanowią podstawę do dalszego projektowania architektury systemu.

### 3.1 Wymagania funkcjonalne

Wymagania funkcjonalne określają zachowania oraz funkcje, jakie system powinien realizować. Na podstawie analizy problemu oraz specyficznych potrzeb użytkowników wyodrębniono następujący zbiór wymagań:

- rejestracja oraz logowanie użytkowników,
- przeglądanie katalogu produktów sklepu komputerowego,
- dodawanie wybranych komponentów do kreatora zestawu PC,
- automatyczna weryfikacja kompatybilności podzespołów, obejmująca między innymi:
  - zgodność gniazda procesora,
  - typ pamięci RAM,
  - kompatybilność częstotliwości i pojemności pamięci,
  - TDP procesora oraz wymagania energetyczne,
  - dopasowanie komponentów do obudowy,
  - kompatybilność układów chłodzenia.
- wykorzystanie logiki rozmytej do klasyfikacji zestawów pod konkretne kategorie (np. budżetowy, wydajny, energooszczędny),

- generowanie gotowych konfiguracji na podstawie wybranych komponentów,
- dodawanie produktów oraz pełnych zestawów do koszyka,
- modyfikacja zawartości koszyka oraz finalizacja zamówienia.

## 3.2 Wymagania niefunkcjonalne

Wymagania niefunkcjonalne opisują właściwości jakościowe systemu, które wpływają na komfort użytkowania, bezpieczeństwo oraz niezawodność.

- **Wydażność:** czas odpowiedzi API nie powinien przekraczać 1 sekundy przy standardowym obciążeniu.
- **Skalowalność:** system powinien umożliwiać uruchomienie w środowisku chmurowym Microsoft Azure.
- **Bezpieczeństwo:** hasła użytkowników muszą być przechowywane w formie zaszyfrowanej (bcrypt).
- **Niezawodność:** system powinien obsługiwać błędy kompatybilności w sposób kontrolowany i komunikatywny.
- **Użyteczność:** interfejs użytkownika powinien być przejrzysty, responsywny i intuicyjny.
- **Przenośność:** frontend uruchamiany w dowolnej nowoczesnej przeglądarce; backend kompatybilny z systemami Linux i Windows.
- **Wysoka dostępność:** system powinien być dostępny w sposób ciągły i niesustanny.

## 3.3 Przypadki użycia

Najważniejsze przypadki użycia systemu obejmują następujące scenariusze:

- „Użytkownik rejestruje nowe konto”
- „Użytkownik loguje się do systemu”
- „Użytkownik przegląda dostępne komponenty”
- „Użytkownik buduje własny zestaw PC”
- „System sprawdza kompatybilność elementów”
- „Użytkownik dodaje zestaw do koszyka”
- „Użytkownik realizuje zamówienie”

## 3.4 Narzędzia i technologie

Realizacja systemu wymagała zastosowania nowoczesnych technologii backendowych, frontendowych oraz narzędzi modelowania. Wszystkie zastosowane narzędzia i technologie przedstawiono w kolejnych podrozdziałach.

### 3.4.1 Frontend

- **React** [react] - biblioteka wykorzystywana do tworzenia interaktywnych interfejsów użytkownika.
- **TypeScript** [typescript] – rozszerzenie JavaScript umożliwiające statyczne typowanie.
- **Vite** [vite] – bundler odpowiedzialny za budowanie i szybkie uruchamianie aplikacji.
- **Tailwind CSS** [tailwind] – framework ułatwiający projektowanie responsywnego interfejsu.
- **Node.js** [nodejs] – środowisko wykonawcze dla narzędzi JavaScript.
- **npm** [npm] – menedżer pakietów odpowiedzialny za instalację zależności.
- **ESLint** [eslint] – narzędzie do analizy i kontroli jakości kodu TypeScript i JavaScript.

### 3.4.2 Backend

- **Python 3** [python] – język implementacji warstwy serwerowej.
- **FastAPI** [fastapi] – framework umożliwiający tworzenie wydajnego REST API.
- **SQLAlchemy** [sqlalchemy] – ORM realizujący komunikację z bazą danych.
- **PostgreSQL** [postgresql] – relacyjna baza danych przechowująca informacje o produktach i zamówieniach.
- **psycopg2** [psycopg2] – sterownik PostgreSQL wykorzystywany do komunikacji z bazą.
- **Pydantic** [pydantic] – walidacja modeli danych wykorzystywana przez FastAPI.
- **Uvicorn** [uvicorn] – serwer ASGI obsługujący zapytania HTTP.

### 3.4.3 Modelowanie i narzędzia wspomagające

- **Oracle SQL Developer Data Modeler** [oracle\_datamodeler] – narzędzie użyte do projektowania modelu ERD bazy danych.
- **Git** [git] – system kontroli wersji.
- **GitHub** [github] – repozytorium kodu oraz platforma do zarządzania wersjami projektu.
- **Microsoft Azure** [azure] – środowisko do wdrożenia i hostowania bazy danych.

### 3.4.4 Modelowanie i narzędzia wspomagające

- **Oracle SQL Developer Data Modeler** - narzędzie użyte do projektowania modelu ERD bazy danych.
- **Git** - system kontroli wersji.
- **GitHub** - repozytorium kodu oraz platforma do zarządzania wersjami projektu.
- **Microsoft Azure** - środowisko do wdrożenia i hostowania bazy danych.

## 3.5 Metodyka pracy

Prace nad projektem prowadzono w sposób iteracyjny, bazując na podejściu zbliżonym do metodyki Agile. Każdy cykl rozwoju obejmował:

- analizę oraz doprecyzowanie wymagań pod standardy rynkowe,
- projekt danej części funkcjonalnych systemu,
- implementację odpowiednio nowych modułów backendu i frontendu,
- testowanie, w przeznaczonym do tego środowisku, poprawności działania,
- refaktoryzację oraz optymalizację kodu.

Takie podejście umożliwiło równoległe rozwijanie bazy danych, logiki serwera i interfejsu użytkownika, a także szybkie reagowanie na pojawiające się problemy i dostosowywanie projektu do bieżących potrzeb oraz wyzwań.



# Rozdział 4

## Specyfikacja zewnętrzna

Celem niniejszego rozdziału jest przedstawienie specyfikacji zewnętrznej zaprojektowanego systemu. Rozdział opisuje wymagania niezbędne do uruchomienia aplikacji, sposób jej instalacji oraz obsługi, a także role użytkowników i aspekty związane z bezpieczeństwem. Przedstawiono również przykładowe scenariusze korzystania z systemu z perspektywy użytkownika końcowego.

### 4.1 Sposób instalacji systemu

System może zostać uruchomiony zarówno w środowisku lokalnym, jak i w środowisku chmurowym. W niniejszej sekcji przedstawiono sposób instalacji oraz uruchomienia aplikacji w obu wariantach.

#### Uruchomienie systemu w środowisku lokalnym

Uruchomienie systemu w środowisku lokalnym polega na lokalnym uruchomieniu warstwy frontendowej oraz backendowej aplikacji. W tym wariancie użytkownik końcowy korzysta z systemu za pośrednictwem przeglądarki internetowej, natomiast połączenie z bazą danych konfigurowane jest poprzez odpowiednie parametry w pliku konfiguracyjnym aplikacji.

W przypadku uruchomienia lokalnego wymagane jest posiadanie na komputerze zainstalowanego interpretera języka Python, środowiska Node.js oraz nowoczesnej przeglądarki internetowej. Warstwa frontendowa aplikacji budowana jest lokalnie przy użyciu skryptów środowiska Node.js, a następnie udostępniana jako aplikacja webowa dostępna pod adresem 127.0.0.1. Backend systemu uruchamiany jest jako serwer aplikacyjny w języku Python, który realizuje logikę biznesową oraz komunikację z bazą danych.

Domyślnie baza danych systemu hostowana jest w środowisku chmurowym, jednak w wariancie lokalnym możliwe jest również podłączenie do innej instancji bazy danych poprzez zmianę adresu IP oraz parametrów połączenia w pliku konfiguracyjnym aplikacji.

Takie rozwiązanie umożliwia elastyczne testowanie systemu bez konieczności utrzymywania lokalnej instancji bazy danych.

### **Uruchomienie systemu w środowisku chmurowym Azure**

Alternatywnie system może zostać uruchomiony w całości w środowisku chmurowym Microsoft Azure. W tym wariantcie baza danych została wdrożona jako usługa Azure Database for PostgreSQL, natomiast warstwa backendowa aplikacji uruchamiana jest jako usługa serwerowa w ramach platformy Azure.

Proces wdrożenia obejmuje utworzenie odpowiednich zasobów chmurowych, takich jak instancja bazy danych oraz środowisko uruchomieniowe dla aplikacji backendowej. Po zakończeniu procesu wdrożenia system dostępny jest publicznie poprzez przeglądarkę internetową, bez konieczności instalacji dodatkowego oprogramowania po stronie użytkownika.

Takie podejście umożliwia uruchomienie aplikacji w środowisku zbliżonym do produkcyjnego oraz weryfikację poprawności jej działania w warunkach rzeczywistych.

## **4.2 Kategorie użytkowników**

Zaprojektowany system przewiduje istnienie kilku kategorii użytkowników, różniących się zakresem dostępnych funkcjonalności. Podział na role użytkowników umożliwia logiczne rozgraniczenie uprawnień oraz odpowiednie zarządzanie dostępem do poszczególnych elementów systemu.

Role użytkowników zostały zdefiniowane w następujący sposób:

- Podstawową kategorią użytkowników są użytkownicy niezalogowani, którzy mają możliwość przeglądania katalogu dostępnych produktów oraz zapoznania się z funkcjonalnością systemu. Użytkownicy ci nie posiadają jednak możliwości zapisywania zestawów ani realizacji zamówień.
- Drugą kategorią są użytkownicy zarejestrowani. Po zalogowaniu uzyskują oni dostęp do rozszerzonej funkcjonalności systemu, obejmującej możliwość tworzenia i zapisywania zestawów komputerowych, dodawania produktów do koszyka oraz składania zamówień. Dane użytkowników są przechowywane w bazie danych systemu.
- System przewiduje również istnienie roli administracyjnej, przeznaczonej do zarządzania zawartością systemu oraz jego konfiguracją. Rola administratora wykorzystywana jest w celach zarządzania danymi systemowymi, takimi jak katalog produktów oraz struktura bazy danych, i jest ograniczona do właściciela systemu.

Zastosowany podział na kategorie użytkowników pozwala na zachowanie porządku w strukturze systemu oraz stanowi podstawę do dalszej rozbudowy aplikacji w przyszłości.



## 4.3 Sposób obsługi systemu

Obsługa systemu została zaprojektowana w sposób intuicyjny i niewymagający specjalistycznej wiedzy technicznej. Podstawowe etapy korzystania z systemu przedstawiono poniżej:

- Użytkownik uruchamia aplikację za pośrednictwem przeglądarki internetowej i uzyskuje dostęp do strony głównej systemu.
- System umożliwia przeglądanie katalogu dostępnych komponentów komputerowych oraz ich filtrowanie zgodnie z preferencjami użytkownika.
- Użytkownik przechodzi do kreatora zestawów komputerowych, w którym dodaje kolejne podzespoły do konfigurowanego zestawu.
- Podczas konfiguracji kreator na bieżąco analizuje zgodność wybieranych komponentów z wcześniej dodanymi elementami zestawu.
- W przypadku wykrycia niezgodności technicznej dalsza realizacja konfiguracji zostaje zablokowana, a użytkownik otrzymuje czytelny komunikat informujący o przyczynie problemu.
- Kreator uniemożliwia utworzenie zestawu niekompatybilnego technicznie, co eliminuje ryzyko popełnienia błędu przez użytkownika.
- Po skompletowaniu poprawnego zestawu użytkownik może zapisać konfigurację lub dodać ją do koszyka.
- Ostatnim etapem jest realizacja zamówienia, obejmująca podsumowanie wybranych komponentów oraz zatwierdzenie zamówienia.

## 4.4 Administracja systemem

Administracja systemem obejmuje czynności związane z zarządzaniem konfiguracją oraz danymi aplikacji. Funkcje administracyjne są ograniczone i przeznaczone do użytku przez właściciela systemu.

Zakres administracji systemem obejmuje w szczególności:

- zarządzanie katalogiem produktów oraz ich parametrami technicznymi przechowywanymi w bazie danych,
- nadzór nad poprawnością struktury danych oraz spójnością informacji wykorzystywanych przez mechanizm weryfikacji kompatybilności,

- aktualizację danych konfiguracyjnych systemu, w tym parametrów połączenia z bazą danych,
- kontrolę poprawności działania aplikacji backendowej oraz jej komunikacji z warstwą frontendową,
- wykonywanie czynności administracyjnych związanych z utrzymaniem systemu w środowisku lokalnym lub chmurowym.

## 4.5 Kwestie bezpieczeństwa

W trakcie projektowania systemu uwzględniono podstawowe aspekty bezpieczeństwa, mające na celu ochronę danych użytkowników oraz zapewnienie poprawnego i przewidywalnego działania aplikacji.

Najważniejsze zagadnienia związane z bezpieczeństwem systemu obejmują:

- przechowywanie danych użytkowników w relacyjnej bazie danych z zachowaniem integralności i spójności informacji,
- hasła użytkowników przechowywane są w bazie danych w postaci skrótów kryptograficznych wygenerowanych z wykorzystaniem algorytmu bcrypt, co uniemożliwia ich odtworzenie w postaci jawnej,
- walidację danych przekazywanych pomiędzy frontendem a backendem w celu ograniczenia ryzyka wprowadzenia niepoprawnych danych,
- ograniczenie dostępu do funkcji administracyjnych wyłącznie do roli administratora systemu,
- brak przechowywania wrażliwych danych po stronie klienta.

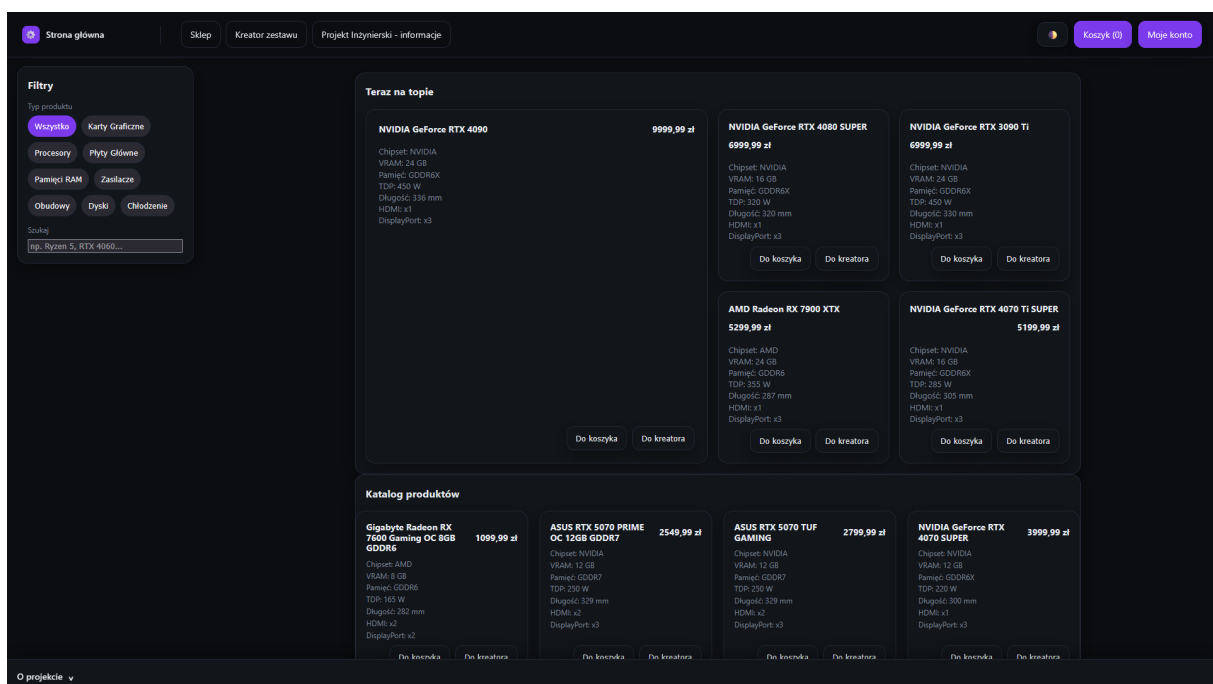
## 4.6 Scenariusze korzystania z systemu

W niniejszej sekcji przedstawiono przykładowe scenariusze korzystania z systemu z perspektywy użytkownika końcowego. Scenariusze ilustrują działanie kluczowych funkcjonalności aplikacji, w tym przeglądanie katalogu produktów, filtrowanie komponentów, zarządzanie koszykiem, proces logowania oraz działanie kreatora zestawów komputerowych. Każdy scenariusz został zilustrowany odpowiednimi zrzutami ekranu przedstawiającymi rzeczywiste działanie systemu.

### 4.6.1 Scenariusz 1: Przeglądanie katalogu komponentów

Użytkownik uruchamia aplikację i uzyskuje dostęp do strony głównej systemu, na której możliwe jest przeglądanie katalogu dostępnych komponentów komputerowych oraz zapoznanie się z ich podstawowymi parametrami technicznymi i ceną.

Na rysunku 4.1 przedstawiono widok strony głównej aplikacji wraz z listą dostępnych produktów.

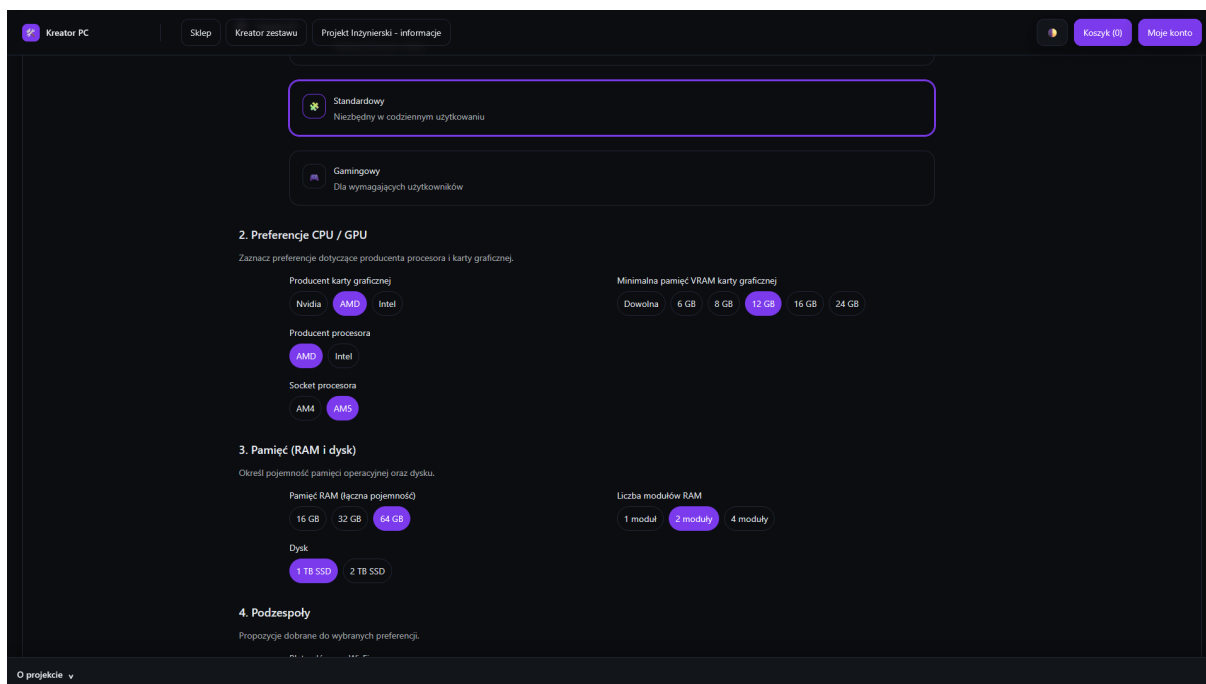


Rysunek 4.1: Widok strony głównej aplikacji

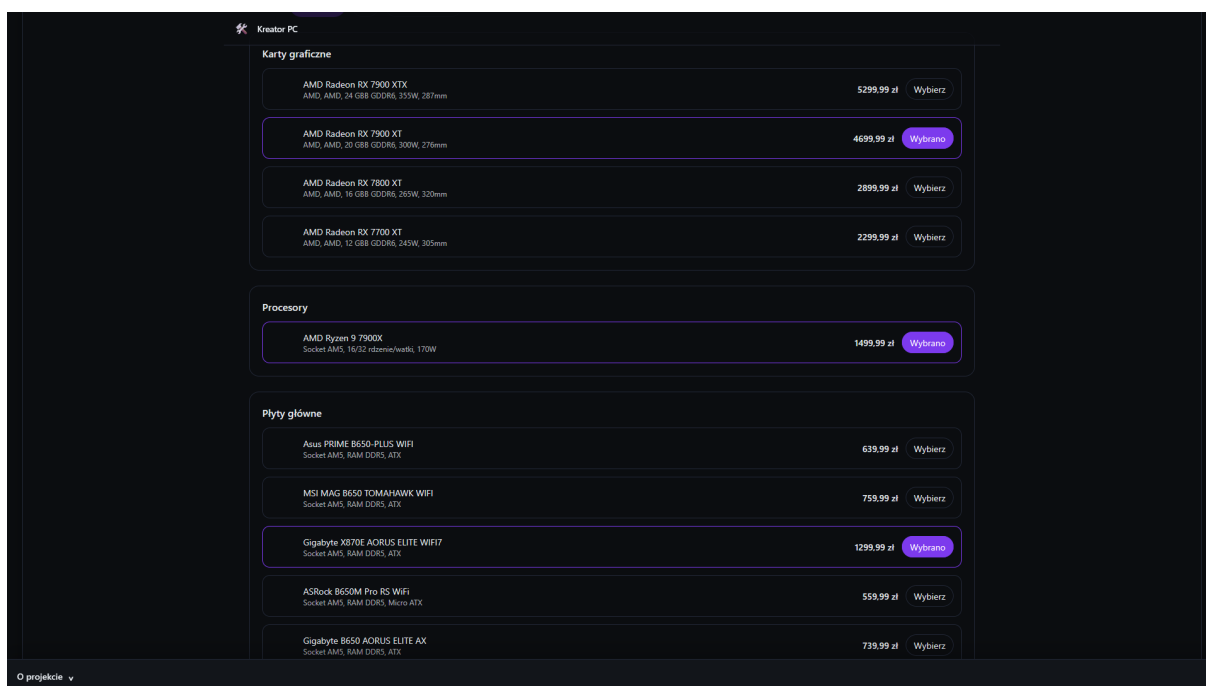
### 4.6.2 Scenariusz 2: Filtrowanie oraz wybór produktów

Użytkownik korzysta z dostępnych mechanizmów filtrowania produktów, takich jak typ komponentu, producent lub parametry techniczne, w celu zawężenia listy wyświetlanych elementów. Następnie wybiera interesujący go produkt z przefiltrowanej listy.

Proces filtrowania produktów przedstawiono na rysunku 4.2, natomiast wybór z komponentów z pośród przefiltrowanej listy na rysunku 4.3.



Rysunek 4.2: Filtrowanie produktów z katalogu

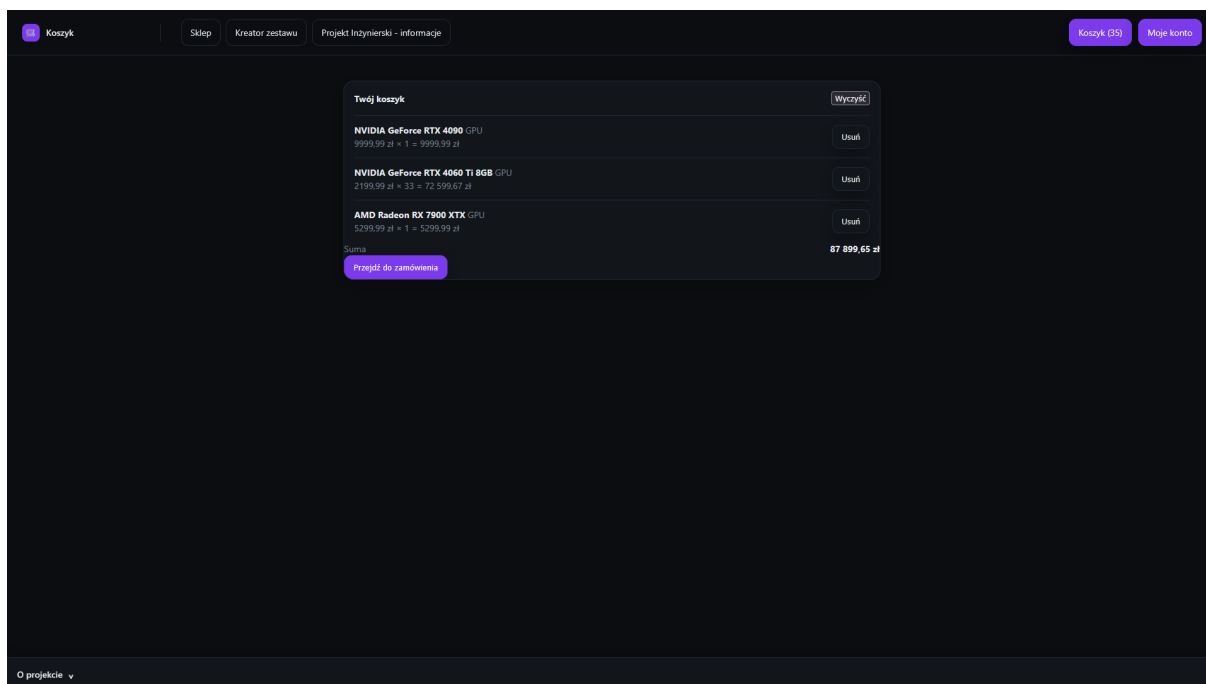


Rysunek 4.3: Wybór produktów z przefiltrowanego katalogu

### 4.6.3 Scenariusz 3: Dodawanie produktów do koszyka

Po wybraniu produktu użytkownik dodaje go do koszyka za pomocą odpowiedniego przycisku dostępnego w widoku produktu. System zapisuje wybrany element w koszyku oraz wyświetla komunikat potwierdzający poprawne wykonanie operacji.

Na rysunku 4.4 przedstawiono widok koszyka zawierającego dodane produkty.

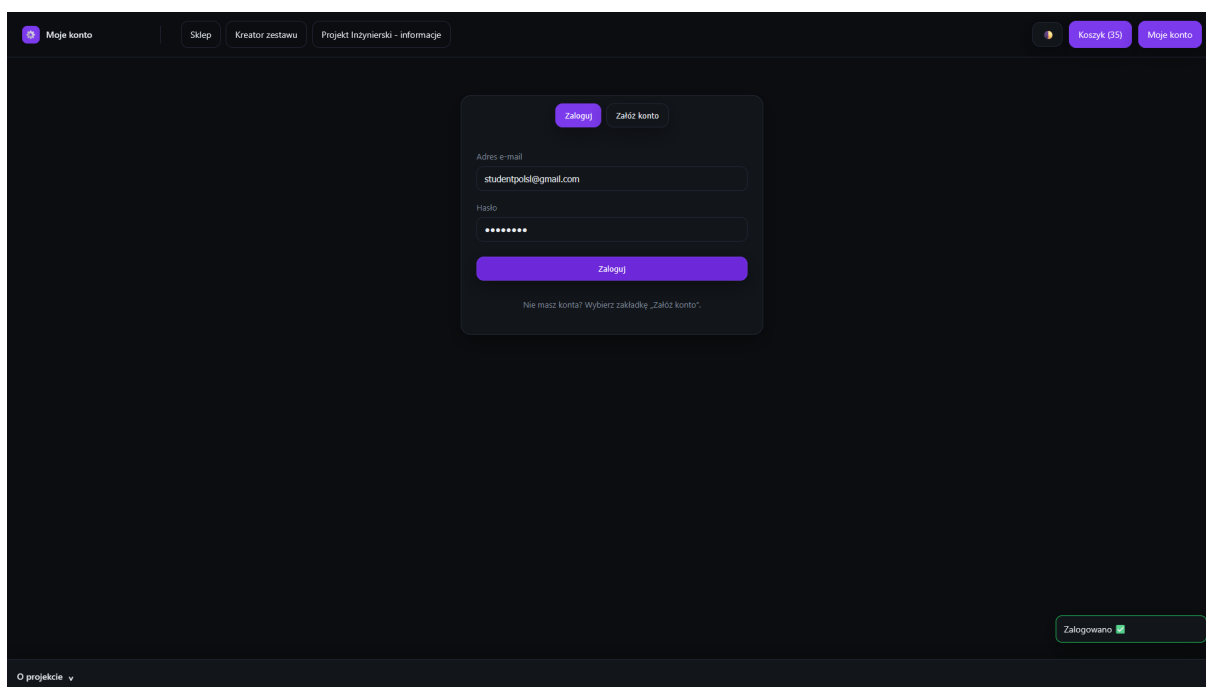


Rysunek 4.4: Widok koszyka użytkownika z dodanymi produktami

#### 4.6.4 Scenariusz 4: Poprawne logowanie użytkownika

Użytkownik przechodzi do widoku logowania i wprowadza poprawne dane uwierzytelniające. Po ich weryfikacji system umożliwia dostęp do funkcji wymagających autoryzacji, takich jak zapis zestawu czy zarządzanie koszykiem.

Widok poprawnego procesu logowania przedstawiono na rysunku 4.5.

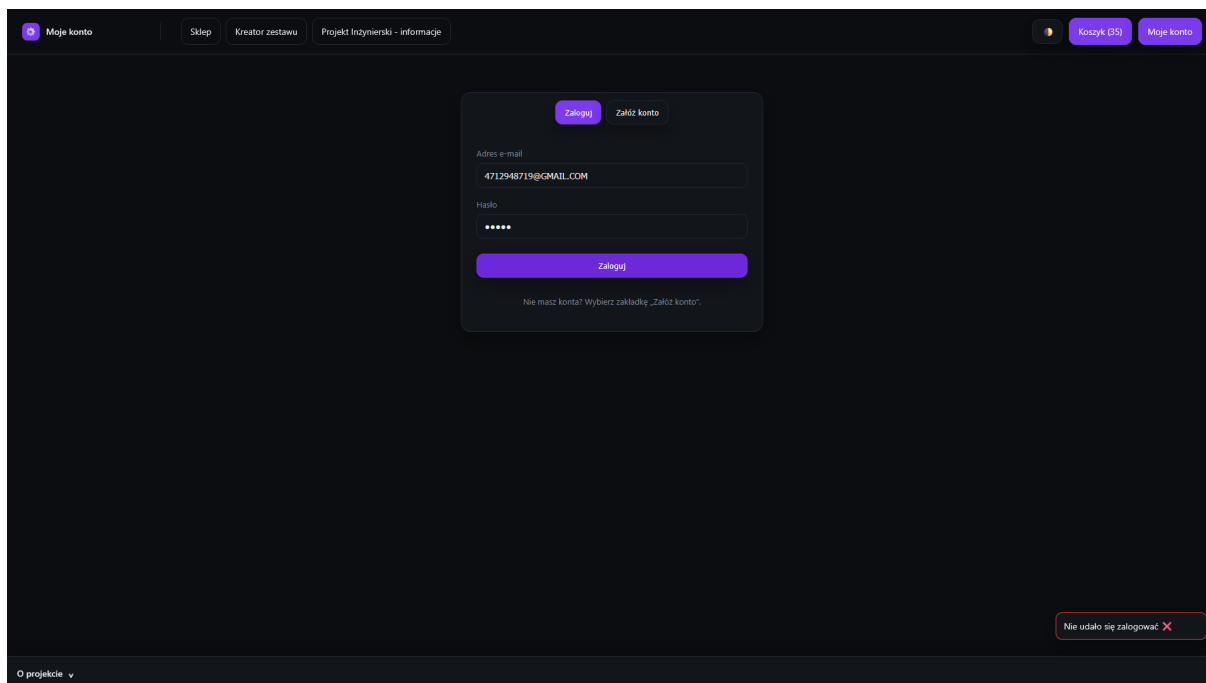


Rysunek 4.5: Poprawne logowanie użytkownika

### 4.6.5 Scenariusz 5: Niepoprawne logowanie użytkownika

W przypadku wprowadzenia niepoprawnych danych logowania system odrzuca próbę uwierzytelnienia i wyświetla stosowny komunikat informujący o błędzie.

Sytuację tę przedstawiono na rysunku 4.6.

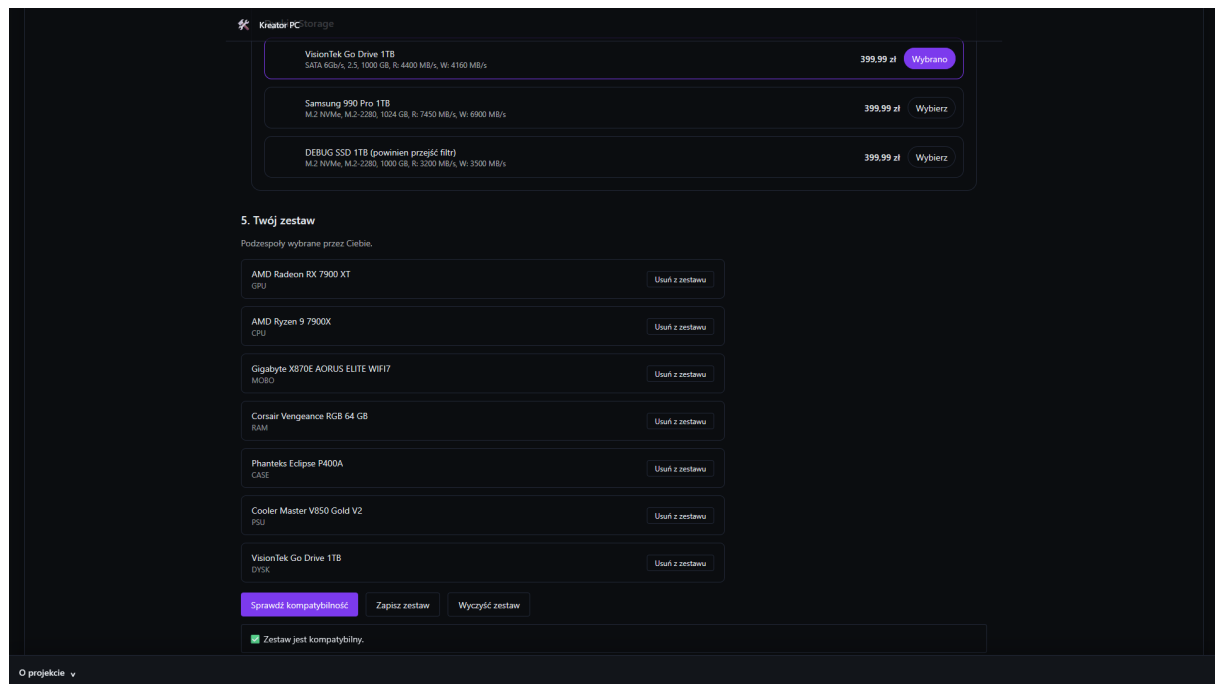


Rysunek 4.6: Komunikat o niepoprawnych danych logowania

### 4.6.6 Scenariusz 6: Utworzenie kompatybilnego zestawu komputerowego

Użytkownik przechodzi do kreatora zestawów komputerowych i wybiera kompatybilne podzespoły, takie jak procesor, płyta główna, pamięć RAM oraz pozostałe elementy konfiguracji. System na bieżąco weryfikuje zgodność techniczną wybieranych komponentów.

Po skompletowaniu poprawnego zestawu użytkownik może zapisać konfigurację lub dodać ją do koszyka. Widok poprawnie skonfigurowanego zestawu przedstawiono na rysunku 4.7.

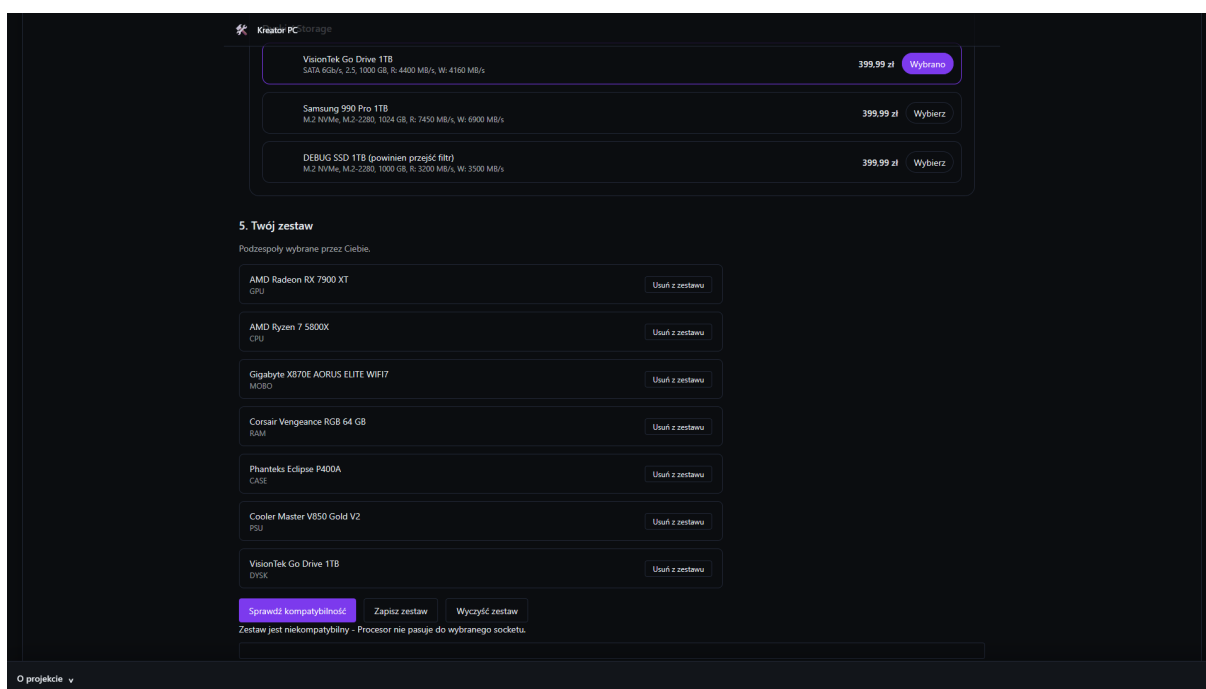


Rysunek 4.7: Widok kreatora zestawów komputerowych

### 4.6.7 Scenariusz 7: Próba utworzenia niekompatybilnego zestawu

W tym scenariuszu użytkownik próbuje skonfigurować zestaw komputerowy zawierający niezgodne technicznie podzespoły, na przykład procesor oraz płytę główną obsługujące różne typy gniazd.

System wykrywa niezgodność i blokuje możliwość zapisania konfiguracji, wyświetlając komunikat informujący o przyczynie problemu, co przedstawiono na rysunku 4.8.



Rysunek 4.8: Komunikat o niezgodności podzespołów



# Rozdział 5

## Specyfikacja wewnętrzna

Celem niniejszego rozdziału jest przedstawienie projektu oraz architektury zaprojektowanego systemu. Zaprezentowano w nim strukturę systemu, podział na warstwy, model danych, architekturę backendu i frontendu oraz zastosowane rozwiązania techniczne.

### 5.1 Przedstawienie idei systemu

System jest odpowiedzią na nieustannie poszerzający się rynek podzespołów komputerowych, pełniąc funkcję inteligentnego asystenta przy kompletowaniu osobistego zestawu PC. Implementacja logiki rozmytej miała na celu ułatwienie nawet początkującym użytkownikom skuteczne przejście przez ten proces, uwzględniając ich potrzeby takie jak wydajność, energooszczędność czy odpowiedni koszt. System łączy intuicyjny interfejs, znany z innych serwisów internetowych, z zaawansowanymi mechanizmami wspomagającymi, zaprojektowanymi konkretnie pod potrzeby klientów.

#### 5.1.1 Cel powstania aplikacji

Celem systemu było uproszczenie samodzielnego składania zestawów komputerowych poprzez inteligentną asekurację podczas tego procesu. Jego celem jest eliminacja jak potencjalnie jak największej liczby błędów po stronie użytkowników.

Aplikację projektowano z myślą o:

- intuicyjnym wyborze komponentów z katalogu,
- automatycznej weryfikacji kompatybilności,
- wsparciu w dopasowaniu sprzętu do potrzeb, używając logiki rozmytej,
- możliwości zapisaniu kompatybilnego zestawu oraz o jego zamówieniu.

### 5.1.2 Innowacyjność systemu

System powstał jako inteligentny kreator zestawów komputerowych, składający się z mechanizmów weryfikacji oraz wspomagania decyzji, podczas gdy podobnych rozwiązań nie sposób znaleźć w najpopularniejszych polskich sklepach komputerowych.

Innowacyjność systemu wynika z:

- automatycznej weryfikacji kompatybilności, obejmującej wszystkie powszechne znane parametry, takie jak:
  - socket procesora i płyty głównej,
  - typ pamięci RAM,
  - format obudowy,
  - maksymalna suma obciążenia dla konkretnego zasilania,
  - fizyczne wymiary elementów,
  - kompatybilność portów, gniazd oraz złącz.
- zastosowania logiki rozmytej, umożliwiającej dobranie odpowiedniego zestawu pod indywidualne potrzeby, dzieląc je na:
  - budżet,
  - wydajność,
  - energooszczędność,
  - przeznaczenie.
- zapewnionego nieustannego monitorowania stanu aplikacji, dbając o wysoką dostępność serwisu i jego wydajność,
- potencjału rozbudowywania aplikacji w rzeczywistym środowisku produkcyjnym, poprzez dbanie o skalowalność i integralność z najnowszymi technologiami.

## 5.2 Architektura systemu

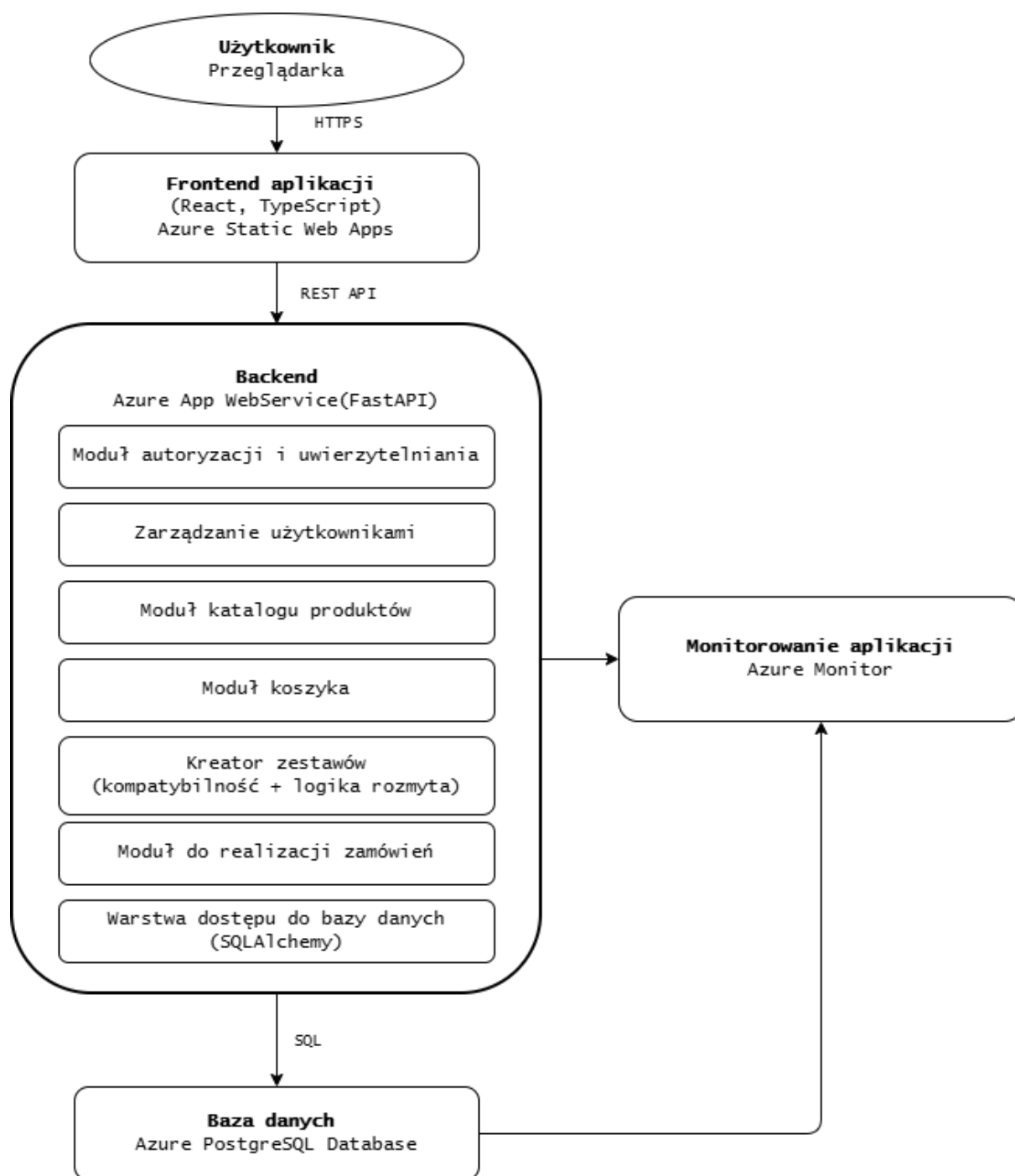
Sekcja ta skupia się na szczegółowej analizie architektury systemu teleinformatycznego, przedstawiając schemat ideowy działania aplikacji oraz omawiając sposób współpracy poszczególnych komponentów systemu.

### 5.2.1 Schemat ideowy

Na rysunku 5.1 przedstawiono schemat ideowy działania zaprojektowanego systemu. Diagram obrazuje podział aplikacji na warstwę frontendową, backendową oraz warstwę bazy danych, a także sposób komunikacji pomiędzy tymi elementami.

Warstwa frontendowa odpowiedzialna jest za interakcję z użytkownikiem oraz prezentację danych i została szczegółowo opisana w rozdziale 3. Warstwa backendowa realizuje logikę biznesową systemu, w tym mechanizmy weryfikacji kompatybilności podzespołów, obsługę koszyka oraz proces uwierzytelniania użytkowników, co omówiono w rozdziale 5. Struktura oraz projekt bazy danych, stanowiącej centralny element przechowywania informacji, zostały przedstawione w rozdziale 4.

Schemat ideowy pozwala na całościowe spojrzenie na architekturę systemu oraz ułatwia zrozumienie zależności pomiędzy poszczególnymi modułami aplikacji, które zostały szczegółowo omówione w kolejnych częściach pracy.



Rysunek 5.1: Schemat ideowy działania aplikacji

### 5.2.2 Frontend

Frontend aplikacji pełni ważną funkcję w funkcjonowaniu każdego systemu, ponieważ odpowiada za prezentację danych dostarczonych z backendu. Projekt wykorzystuje nowoczesne technologie React, TypeScript oraz Vite, zapewniając innowacyjność, wydajność oraz skalowalność. Inteligentny kreator zestawów komputerowych to jednostronicowy typ aplikacji webowej, dzięki czemu przełączanie widoku następuje dynamicznie bez konieczności ponownego ładowania zasobów. Takie rozwiązanie gwarantuje wysoką wydajność, prostotę oraz komfort użytkowania.

Zadania realizowane przez interfejs:

- prezentacja katalogu produktów, pobieranych dynamicznie z backendu,
- wybranie komponentów do przeniesienia z katalogu do kreatora,
- obsługa koszyka, dodawanie, modyfikowanie i usuwanie elementów,
- reakcja na działania użytkownika wykorzystując React Hooks,
- wyświetlanie aktualnego stanu kompatybilności zestawów,
- wykonywanie zapytań REST API do backendu w celu weryfikacji danych i aktualizacji stanu aplikacji.

Za wygląd zgodny ze standardami rynkowymi odpowiedzialny jest Tailwind CSS, gwarantując responwność i estetykę rozwiązania. Framework umożliwia konfigurację i nadawanie stylów bezpośrednio w komponentach poprzez gotowe klasy utility. Takie rozwiązanie zapewnia intuicyjne i dynamiczne UI. Za komunikację z backendem odpowiada REST API poprzez zapytania realizowane biblioteką fetch API. Każda interakcja ze stroną powoduje wywołanie właściwego endpoint'u, co dynamicznie aktualizuje stan aplikacji, w tym bazę danych. Projekt wdrożony został w chmurze Microsoft Azure, jako Azure Static Web App, pełniącej rolę hostingu. Takie rozwiązanie umożliwia automatyzację CI/CD, co przekłada się na bezproblemowość i stabilność. Całość stacku technologicznego zapewnia wysoką dostępność aplikacji oraz łatwość korzystania i zarządzania.

Projekt frontendu powstał z myślą o:

- przejrzystości interfejsu,
- prostocie korzystania,
- szybkości działania.

Takie podejście powoduje, że aplikacja jest łatwa w utrzymaniu, skalowalna, wysoko dostępna oraz przejrzysta zarówno dla początkujących jak i zaawansowanych użytkowników.

### 5.2.3 Backend

Backend, jako jeden z filarów każdej aplikacji, odpowiada za logikę projektu, przetwarzanie danych oraz za kluczową komunikację kodu z bazą danych. Implementacja w języku Python, z wykorzystaniem frameworka FastAPI zapewnia nowoczesność i wydajność usług sieciowych, gwarantując przejrzystość, skalowalność oraz integralność z frontendem. Backend pełni funkcję warstwy pośredniczącej pomiędzy interfejsem a bazą danych, gdzie żądania od strony użytkownika są nieustannie przetwarzane, a następnie przekazywane do odpowiednich modułów. Zwracana odpowiedź, w formacie JSON, umożliwia płynną komunikację w architekturze aplikacji.

Struktura omawianej warstwy zawiera następujące moduły funkcjonalne:

- moduł autoryzacji i uwierzytelniania - odpowiada za rejestrację, logowanie i odpowiednie weryfikowanie danych użytkownika. Wykorzystuje algorytm bcrypt do zapewnienia bezpieczeństwa haseł,
- moduł zarządzania użytkownikami - umożliwia monitorowanie historii zamówień,
- moduł katalogu produktów - umożliwia przeglądanie oferty sklepu, pobieranie listy części, filtrowanie i sortowanie,
- moduł kreatora - zapewnia możliwość zbierania danych o wybranych komponentach, gromadząc je w strukturze lokalnej sesji użytkownika, pozwalając na transport informacji z oferty do kreatora,
- moduł weryfikacji kompatybilności - odpowiada za porównywanie parametrów elementów zestawu, analizując zgodność pomiędzy nimi, stanowiąc rdzeń i główną funkcjonalność projektu,
- moduł logiki rozmytej - odpowiedzialny jest za klasyfikację zestawu do jednej z kategorii,
- moduł koszyka i zamówień - zadaniem tego modułu jest dodawanie elementów do koszyka, nieustanne monitorowanie jego stanu i modyfikowanie zawartości.

Komunikacja pomiędzy backendem a bazą danych zaprojektowana została przy pomocy SQLAlchemy, który pełni funkcję warstwy ORM. Takie rozwiązanie zapewnia odwzorowanie struktury tabel w postaci obiektów, znacząco ułatwiając implementację logiki, minimalizując ryzyko błędów oraz zwiększając bezpieczeństwo. Operacje wykonywane w ten sposób określamy jako bezpieczne i atomowe. Wdrożenie w postaci Azure App Service, zapewnia skalowalność, prostotę utrzymania oraz wysoką dostępność, co jest niezwykle istotne podczas projektowania aplikacji przeznaczonej dla klientów. Uruchamianie nowych wersji odbywa się automatycznie poprzez mechanizm stałej implementacji i stałego dostarczania, znanego szerzej jako CI/CD.

Backend zaprojektowany został z myślą o:

- wysokiej wydajności,
- przejrzystości kodu,
- bezpieczeństwie danych,
- skalowalności,
- niezawodności,
- wysokiej dostępności.

Realizacja tej części projektu w sposób opisany powyżej stanowi stabilną i skalowalną podstawę działania aplikacji, gwarantując poprawne komunikowanie się żądań użytkownika z bazą danych.

### 5.2.4 Baza danych

Baza danych jest fundamentem każdej aplikacji biznesowej, gdyż odpowiedzialna jest za trwałe przechowywanie informacji o użytkownikach, produktach oraz zamówieniach. Zaprojektowana została w oparciu o zasady normalizacji oraz analizę charakterystycznych właściwości komponentów, zapewniając integralność danych oraz efektywne weryfikowanie kompatybilności elementów.

Model i schemat danych zaprojektowany został przy użyciu Oracle SQL Developer Data Modeler, co umożliwiło utworzenie diagramu ERD oraz automatyczne wygenerowanie definicji tabel. Wyeksportowana baza jako plik DDL wdrożona została do środowiska DataGrip, w którym wypełniono ją przygotowanymi INSERT'ami, zaciągniętymi z publicznego repoztorium komponentów[3], w celu implementacji danych produktów. System operował w oparciu o PostgreSQL, pełniący funkcję środowiska testowego wykorzystywanego do sprawnego implementowania backendu i modułów logiki. Gdy projekt przeszedł do etapu wdrożenia produkcyjnego, baza danych została zmigrowana do chmury Microsoft Azure, korzystając z gotowych poleceń podanych w dokumentacji Microsoft Azure[1]. Przeniesiona struktura zaimplementowana została jako Azure PostgreSQL Database, co dzięki funkcjonalności grupowania zasobów Azure zapewnia integrację z produkcyjną wersją backendu uruchomioną w Azure App Service.

Baza Danych spełnia wymagi trzeciej postaci normalnej (3NF), co oznacza, że nie zawiera redundancji danych, wszystkie atrybuty niekluczowe są w pełni zależne od klucza głównego właściwych tabel. Właściwość ta pozwala na:

- zminimalizowanie ryzyka niespójnych danych,
- zwiększenie wydajności zapytań,
- eliminację duplikatów.

Kluczową rolę w bazie danych pełni tabela Produkty, będąc ogólnym rejestrem wszystkich komponentów dostępnych w ofercie. Przechowuje ona podstawowe informacje o produktach, natomiast szczegółowe parametry znajdują się w dedykowanych tabelach zależnych od typu podzespołu. Takie rozwiązanie eliminuje redundację danych oraz dodaje przejrzystości do przechowywania właściwości.

W celu przechowywania wielu komponentów w jednym zestawie, koszyku lub zamówieniu zastosowano liczne relacje typu M:N, które zostały zrealizowane przy pomocy tabel łączących. Całość struktury została zabezpieczona kluczami obcymi, co zapewnia pełną integralność referencyjną oraz uniemożliwia tworzenie błędnych powiązań między danymi. Modularny charakter modelu umożliwia jego dalszy rozwój, w tym dodawanie nowych typów komponentów.

Dzięki zastosowaniu odpowiednich narzędzi, normalizacji oraz chmurowej infrastruktury Azure, baza danych stanowi stabilny i skalowalny fundament systemu, wspierający zarówno logikę kompatybilności, jak i zarządzanie koszykami, zamówieniami oraz zestawami użytkowników.

### **5.2.5 Infrastruktura chmurowa**

Wybór infrastruktury chmurowej jest podstawowym aspektem każdego nastawionego na wysoką dostępność projektu. Chmura Microsoft Azure gwarantuje skalowalność i bezpieczne środowisko, umożliwiając integrację z poszczególnymi komponentami systemu oraz automatyzację wdrażania kolejnych części aplikacji.

Organizację pracy znacząco upraszcza ulokowanie środowiska produkcyjnego w oparciu o grupę zasobów Azure, która zbiera całość w jednej strukturze administracyjnej. Takie rozwiązanie gwarantuje intuicyjne i kontrolowane zarządzanie usługami, co pozwala nieprzerwanie monitorować zużycie zasobów oraz koszty serwisu.



Projekt operował na kredytach Azure o równowartości 100 USD, przyznanych przez Microsoft w ramach programu Azure for Students, pozwalając na wdrożenie aplikacji bez generowania dodatkowych kosztów. Środki te umożliwiły uruchomienie wszystkich wymaganych usług, takich jak:

- Azure App Service,
- Azure Static Web App,
- Azure Database for PostgreSQL.

### **Azure Static Web App**

Warstwa frontend'u zaimplementowana została w usłudze Static Web Apps, co umożliwiło prezentowanie strony w sieci Internet. Usługa ta obsługuje:

- automatyzację CI/CD,
- integrację z backendem.

Stanowi to lekkie, skalowalne rozwiązanie, zapewniając obsługę wielu użytkowników jednocześnie.

### **Azure App Service**

Warstwa backend'u została wdrożona przy użyciu usługi App Service. Oparty na frameworku FastAPI i języku Python osadzona została jako aplikacja, działając na serwerze automatyzowanym przez usługi Azure, zapewniając dzięki temu:

- uproszczone wdrażanie aktualizacji,
- zarządzanie środowiskiem wykonawczym,
- integrację z usługami monitorującymi Azure,
- stabilność,
- wysoką dostępność.

Takie rozwiązanie uodparnia backend na obciążenia i znacząco redukuje wkład użytkownika w administrację w czasie rzeczywistym.

## Azure Database for PostgreSQL

Dane o podzespołach przetrzymywane są w usłudze Azure Database for PostgreSQL, zapewniając środowisko do zarządzania relacyjnymi bazami danych. Wdrożenie polegało na zmigrowaniu bazy PostgreSQL do Azure [1], obejmując elementy takie jak:

- eksport struktury i danych,
- dostosowanie typów danych,
- weryfikację integralności.

Rozwiązanie to zapewnia dostęp do bazy z dowolnego urządzenia, gwarantując elastyczność pracy i wysoką niezawodność. Przechowywanie danych w chmurze daje możliwość elastycznego dostępu do danych zależnie od potrzeb, redukując koszty utrzymania.

## Azure Monitor

Aplikacja jest monitorowana w czasie rzeczywistym za pomocą usług:

- Azure Monitor,
- Application Insights.

Takie rozwiązanie umożliwia nieustanne wykrywanie problemów, reagowanie na zwiększony ruch oraz automatyzację dopasowywania zasobów do aktualnych potrzeb.

## Koszty i optymalizacja

W trakcie wdrażania przeanalizowany został potencjalny koszt utrzymania aplikacji, dobierając odpowiednie rozwiązania, o możliwie niskich parametrach z uwagi na niski poziom ruchu na stronie. Wprowadzone modyfikacje nie mają wpływu na funkcjonalność, pozwalają one na oszczędność środków, których zużycie nawet na średnich komponentach przekracza założony budżet. Wybrane do tego zostały następujące rozwiązania:

- wybranie maszyny wirtualnej o najniższych możliwych parametrach,
- ustawienie automatycznego usypiania systemu w przypadku ruchu poniżej progu zdefiniowanego ręcznie,
- ograniczenie ruchu sieciowego do testowej podsiatki wirtualnej.

W ten sposób możliwe stało się zmieszczenie w oferowanych przez Microsoft kredytach, ale również przygotowało projekt do ewentualnych przyszłych modyfikacji.

## Architektura logiczna

Infrastruktura została umieszczona w jednej grupie zasobów, umożliwiając spójne zarządzanie, wzajemny dostęp do zasobów oraz przystępną modyfikację pojedynczych modułów. Działanie infrastruktury przebiega w następujący sposób:

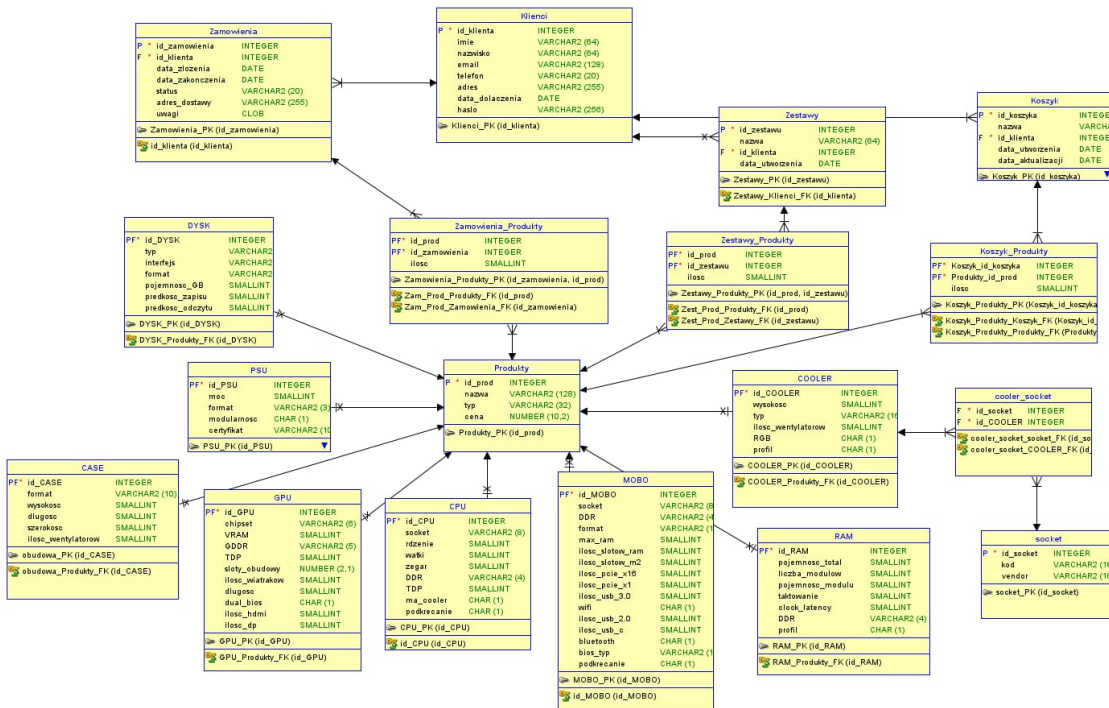
- połączenie użytkownika z frontendem (Static Web App),
- komunikacja na linii frontend - backend (RestAPI),
- komunikacja na linii backend - Baza PostgreSQL,
- monitorowanie komponentów, zwracanie dedykowanych logów, które analizowane przez Azure Monitor dostosowują parametry aplikacji.

Taki sposób implementacji zapewnia wysoką wydajność, stabilność, skalowalność oraz integralność, czyniąc aplikację gotową do działania w produkcyjnym środowisku.

## 5.3 Baza danych

Sekcja ta przedstawia kompletny opis bazy danych użytej w projekcie, opisując szczegółowo wszystkie jej encje, zależności i logikę.

### 5.3.1 Model danych - ERD



Rysunek 5.2: Diagram ERD bazy danych

Na rysunku przedstawiono diagram ERD bazy danych systemu sklepu komputerowego z kreatorem zestawów PC. Model odzwierciedla główne obszary funkcjonalne: aplikacji

- zarządzanie klientami,
- zarządzanie produktami,
- zarządzanie koszykami,
- zarządzanie zamówieniami.

Centralne miejsce w diagramie zajmuje tabela **Produkty**, pełniąca rolę nadrzędnego rejestru wszystkich pozycji dostępnych w systemie. Zawiera ona podstawowe informacje o każdym produkcie, takie jak nazwa, typ oraz cena, natomiast szczegółowe parametry techniczne przechowywane są w dedykowanych tabelach odpowiadających konkretnym kategoriom sprzętu: CPU, GPU, RAM, MOBO, PSU, COOLER, DYSK oraz CASE. Każda

z tych tabel posiada klucz obcy wskazujący na rekord w tabeli Produkty, co pozwala uniknąć redundancji danych oraz ułatwia zarządzanie wspólnymi atrybutami produktów. Relacje związane z procesem zakupowym odwzorowane są poprzez tabele Klienci, Koszyk, Zamówienia oraz Zestawy. Każdy klient może posiadać wiele koszyków, zamówień oraz zapisanych zestawów, co odzwierciedlono relacjami 1:N. Zawartość koszyka, zamówienia oraz zestawu została zrealizowana za pomocą tabel łączących Koszyk\_Produkty, Zamówienia\_Produkty oraz Zestawy\_Produkty, które przechowują pary identyfikatorów produktu i obiektu nadrzędnego wraz z informacją o liczbie sztuk. Umożliwia to poprawne odwzorowanie relacji wiele-do-wielu pomiędzy produktami a koszykami, zamówieniami i zestawami.

Istotnym elementem modelu są tabele socket oraz cooler\_socket, pełniące rolę słowników technicznych wykorzystywanych w logice kompatybilności. Tabela socket przechowuje informacje o typach gniazd procesorów, natomiast tabela cooler\_socket łączy coolery z obsługiwanymi przez nie socketami. Rozwiązanie to umożliwia powiązanie procesorów, płyt głównych oraz systemów chłodzenia oraz ułatwia implementację reguł sprawdzających zgodność komponentów.

Zaprojektowany model danych zachowuje spójność referencyjną dzięki zastosowaniu kluczy głównych i obcych, a rozdzielenie danych ogólnych i szczegółowych na osobne tabele umożliwia utrzymanie bazy w co najmniej trzeciej postaci normalnej. Takie podejście ułatwia dalszą rozbudowę systemu o nowe typy podzespołów oraz dodatkowe funkcje biznesowe, jednocześnie zapewniając integralność i wysoką jakość przechowywanych danych.

Opis przeznaczenia encji:

1. Tabela Klienci przechowuje dane identyfikacyjne użytkowników systemu. Zawiera informacje takie jak imię, nazwisko, adres e-mail, numer telefonu, adres zamieszkania oraz datę dołączenia do systemu. Tabela ta stanowi podstawę identyfikacji użytkowników i jest powiązana z tabelami Koszyki, Zamówienia oraz Zestawy.
2. Tabela Koszyki odpowiada za przechowywanie aktualnych koszyków zakupowych użytkowników. Każdy rekord reprezentuje pojedynczy koszyk przypisany do konkretnego klienta. Tabela zawiera informacje o dacie utworzenia oraz ostatniej aktualizacji koszyka i umożliwia przechowywanie tymczasowych konfiguracji produktów przed złożeniem zamówienia.
3. Tabela Koszyk\_Produkty jest tabelą pośredniczącą realizującą relację wiele-do-wielu pomiędzy tabelami Koszyki oraz Produkty. Przechowuje identyfikatory koszyka i produktu wraz z informacją o aktualnie przechowywanej liczbie sztuk danego

produktu. Rozwiązanie to umożliwia elastyczne i dynamiczne zarządzanie zawartością koszyka.

4. Tabela Zamówienia przechowuje informacje o zamówieniach złożonych przez użytkowników. Zawiera dane takie jak data złożenia zamówienia, data zakończenia realizacji, status zamówienia, adres dostawy oraz dodatkowe uwagi. Każde zamówienie jest przypisane do konkretnego klienta.
5. Tabela Zamówienia\_Produkty realizuje relację wiele-do-wielu pomiędzy tabelami Zamówienia oraz Produkty. Przechowuje informacje o produktach wchodzących w skład zamówienia oraz ich liczbach, umożliwiając poprawne odwzorowanie struktury zamówienia.
6. Tabela Zestawy przechowuje konfiguracje zestawów komputerowych tworzone przez użytkowników. Każdy zestaw posiada nazwę, datę utworzenia oraz jest przypisany do konkretnego klienta. Tabela ta umożliwia zapisywanie i ponowne wykorzystanie konfiguracji zestawów.
7. Tabela Zestawy\_Produkty jest tabelą pośredniczącą odwzorowującą relację wiele-do-wielu pomiędzy tabelami Zestawy oraz Produkty. Przechowuje identyfikatory zestawu i produktu wraz z informacją o liczbie sztuk, co pozwala na budowę pełnej konfiguracji zestawu komputerowego.

### 5.3.2 Opis najważniejszych encji

#### Tabela produkty

Najbardziej istotną, pełniącą funkcję głównej, tabelą w całej bazie danych jest tabela produkty. Kluczem tabeli jest `id_prod`, klucz ten podstawą działania aplikacji. Encja produkty zawiera również pełną nazwę produktu w typie `varchar`, typ produktu, który umożliwia prowadzenie poprawnych operacji w kreatorze zestawów, stanowiąc pole odpowiedzialne za łączenie odpowiednich tabel z dodatkowymi danymi konkretnego produktu, zależnie od jego rodzaju. Tabela zawiera również kolumnę `cena` w typie `numeric`, który wybrano w celu zachowania spójności działania programu. Strukturę tabeli Zestawy przedstawiono w tabeli 5.1.

Tabela 5.1: Struktura tabeli Produkty

| Nazwa pola           | Typ danych                 | Opis                |
|----------------------|----------------------------|---------------------|
| <code>id_prod</code> | <code>integer</code>       | Klucz główny tabeli |
| <code>nazwa</code>   | <code>varchar(128)</code>  | Nazwa produktu      |
| <code>typ</code>     | <code>varchar(32)</code>   | Typ podzespołu      |
| <code>cena</code>    | <code>numeric(10,2)</code> | Cena produktu       |

Poniżej przedstawiono opis poszczególnych pól wchodzących w skład tabeli Produkty:

1. Pole `id_prod` jest kluczem głównym tabeli Produkty. Służy do jednoznacznej identyfikacji każdego produktu w systemie oraz stanowi podstawę powiązań z pozostałymi tabelami bazy danych.
2. Atrybut `nazwa` przechowuje pełną nazwę produktu w postaci tekstowej. Wartość ta jest wykorzystywana w warstwie frontendowej do prezentacji informacji użytkownikowi oraz w backendzie podczas obsługi logiki biznesowej.
3. Pole `typ` określa kategorię produktu, taką jak procesor, karta graficzna czy pamięć RAM. Atrybut ten umożliwia rozróżnienie rodzaju podzespołu oraz poprawne powiązanie rekordu z odpowiednią tabelą zawierającą szczegółowe parametry techniczne.
4. Atrybut `cena` przechowuje aktualną cenę produktu w postaci liczby typu numerycznego. Zastosowanie tego typu danych umożliwia precyzyjne operacje obliczeniowe związane z wyliczaniem wartości koszyka oraz zamówień.

## Tabela Klienci

Tabela Klienci przechowuje dane identyfikacyjne użytkowników systemu oraz informacje niezbędne do realizacji procesów zakupowych. Encja ta stanowi podstawę obsługi użytkowników aplikacji i jest powiązana z tabelami Koszyki, Zamówienia oraz Zestawy. Dane zgromadzone w tabeli umożliwiają jednoznaczną identyfikację klienta oraz przypisanie mu odpowiednich zasobów systemowych. Strukturę tabeli Zestawy przedstawiono w tabeli 5.2.

Tabela 5.2: Struktura tabeli Klienci

| Nazwa pola      | Typ danych   | Opis                           |
|-----------------|--------------|--------------------------------|
| id_klienta      | integer      | Klucz główny tabeli            |
| imie            | varchar(64)  | Imię klienta                   |
| nazwisko        | varchar(64)  | Nazwisko klienta               |
| email           | varchar(128) | Adres poczty elektronicznej    |
| telefon         | varchar(20)  | Numer telefonu kontaktowego    |
| adres           | varchar(255) | Adres zamieszkania klienta     |
| data_dolaczenia | date         | Data rejestracji w systemie    |
| haslo           | varchar(256) | Zaszyfrowane hasło użytkownika |

Poniżej przedstawiono opis poszczególnych pól wchodzących w skład tabeli Klienci:

1. Pole id\_klienta jest kluczem głównym tabeli Klienci. Służy do jednoznacznej identyfikacji użytkownika w systemie oraz stanowi podstawę powiązań z innymi tabelami bazy danych.
2. Atrybut imie przechowuje imię klienta w postaci tekstowej. Informacja ta wykorzystywana jest głównie do celów prezentacyjnych w interfejsie użytkownika.
3. Pole nazwisko zawiera nazwisko klienta. Wraz z polem imie umożliwia pełną identyfikację użytkownika w systemie.
4. Atrybut email przechowuje adres poczty elektronicznej klienta, który wykorzystywany jest jako dane kontaktowe oraz może pełnić rolę identyfikatora podczas procesu logowania.
5. Pole telefon zawiera numer telefonu kontaktowego użytkownika. Pole to umożliwia dodatkową formę kontaktu z klientem w ramach realizacji zamówień.
6. Atrybut adres przechowuje adres zamieszkania klienta, wykorzystywany w procesie realizacji dostawy zamówionych produktów.
7. Pole data\_dolaczenia określa datę rejestracji użytkownika w systemie. Domyślna wartość pola ustawiana jest na aktualną datę w momencie utworzenia rekordu.



8. Atrybut hasło przechowuje hasło użytkownika w postaci zaszyfrowanej. Pole to wykorzystywane jest w procesie uwierzytelniania i zapewnia bezpieczeństwo dostępu do konta użytkownika.

### Tabela Koszyk

Tabela Koszyk przechowuje informacje o koszykach zakupowych użytkowników systemu. Każdy koszyk przypisany jest do konkretnego klienta i służy do tymczasowego gromadzenia wybranych produktów przed złożeniem zamówienia. Encja ta umożliwia zarządzanie zawartością koszyka oraz śledzenie zmian dokonywanych przez użytkownika. Strukturę tabeli Zestawy przedstawiono w tabeli 5.3.

Tabela 5.3: Struktura tabeli Koszyk

| Nazwa pola        | Typ danych  | Opis                                |
|-------------------|-------------|-------------------------------------|
| id_koszyka        | integer     | Klucz główny tabeli                 |
| nazwa             | varchar(64) | Nazwa koszyka                       |
| id_klienta        | integer     | Identyfikator klienta               |
| data_utworzenia   | date        | Data utworzenia koszyka             |
| data_aktualizacji | date        | Data ostatniej aktualizacji koszyka |

Poniżej przedstawiono opis poszczególnych pól wchodzących w skład tabeli Koszyk:

1. Pole id\_koszyka jest kluczem głównym tabeli Koszyk. Służy do jednoznacznej identyfikacji koszyka w systemie oraz stanowi podstawę powiązań z tabelą Koszyk\_Produkty.
2. Atrybut nazwa przechowuje nazwę koszyka, która może być wykorzystywana do rozróżniania wielu koszyków przypisanych do jednego użytkownika.
3. Pole id\_klienta jest kluczem obcym wskazującym na tabelę Klienci. Pole to umożliwia jednoznaczne przypisanie koszyka do konkretnego użytkownika systemu.
4. Atrybut data\_utworzenia określa datę utworzenia koszyka w systemie. Informacja ta może być wykorzystywana do celów analitycznych oraz porządkowych.
5. Pole data\_aktualizacji przechowuje datę ostatniej modyfikacji koszyka. Pole to pozwala na śledzenie aktywności użytkownika oraz zmian wprowadzanych w zawartości koszyka.

## Tabela Zamówienia

Tabela Zamówienia przechowuje informacje dotyczące zamówień składanych przez użytkowników systemu. Każdy rekord reprezentuje pojedyncze zamówienie przypisane do konkretnego klienta i stanowi finalny etap procesu zakupowego. Encja ta umożliwia śledzenie przebiegu realizacji zamówienia oraz przechowywanie danych niezbędnych do jego obsługi. Strukturę tabeli Zestawy przedstawiono w tabeli 5.4.

Tabela 5.4: Struktura tabeli Zamówienia

| Nazwa pola       | Typ danych   | Opis                                   |
|------------------|--------------|----------------------------------------|
| id_zamowienia    | integer      | Klucz główny tabeli                    |
| id_klienta       | integer      | Identyfikator klienta                  |
| data_zlozenia    | date         | Data złożenia zamówienia               |
| data_zakonczenia | date         | Data zakończenia realizacji zamówienia |
| status           | varchar(20)  | Status zamówienia                      |
| adres_dostawy    | varchar(255) | Adres dostawy                          |
| uwagi            | text         | Dodatkowe uwagi do zamówienia          |

Poniżej przedstawiono opis poszczególnych pól tabeli Zamówienia:

1. Pole `id_zamowienia` jest kluczem głównym tabeli Zamówienia. Służy do jednoznacznej identyfikacji zamówienia w systemie oraz stanowi podstawę powiązań z tabelą `Zamówienia_Produkty`.
2. Atrybut `id_klienta` jest kluczem obcym wskazującym na tabelę `Klienci`. Umożliwia przypisanie zamówienia do konkretnego użytkownika systemu.
3. Pole `data_zlozenia` określa datę złożenia zamówienia przez klienta. Informacja ta wykorzystywana jest do monitorowania czasu realizacji zamówienia.
4. Atrybut `data_zakonczenia` przechowuje datę zakończenia realizacji zamówienia. Pole to pozwala na określenie momentu finalizacji procesu zakupowego.
5. Pole `status` określa aktualny stan zamówienia, na przykład złożone, w trakcie realizacji lub zakończone. Umożliwia to kontrolę przebiegu realizacji zamówień.
6. Atrybut `adres_dostawy` przechowuje adres, na który mają zostać dostarczone zamówione produkty. Informacja ta jest niezbędna do poprawnej realizacji procesu logistycznego.
7. Pole `uwagi` umożliwia zapisanie dodatkowych informacji lub komentarzy dotyczących zamówienia, przekazanych przez użytkownika.

## Tabela Zestawy

Tabela Zestawy przechowuje informacje o zestawach komputerowych tworzonych przez użytkowników systemu. Każdy rekord reprezentuje pojedynczy zestaw przypisany do konkretnego klienta i umożliwia zapisywanie oraz ponowne wykorzystanie konfiguracji podzespołów. Encja ta stanowi element kreatora zestawów komputerowych. Strukturę tabeli Zestawy przedstawiono w tabeli 5.5.

Tabela 5.5: Struktura tabeli Zestawy

| Nazwa pola      | Typ danych  | Opis                    |
|-----------------|-------------|-------------------------|
| id_zestawu      | integer     | Klucz główny tabeli     |
| nazwa           | varchar(64) | Nazwa zestawu           |
| id_klienta      | integer     | Identyfikator klienta   |
| data_utworzenia | date        | Data utworzenia zestawu |

Poniżej przedstawiono opis poszczególnych pól tabeli Zestawy:

1. Atrybut `id_zestawu` jest kluczem głównym tabeli Zestawy. Służy do jednoznacznej identyfikacji zestawu w systemie oraz stanowi podstawę powiązań z tabelą `Zestawy_Produkty`.
2. Atrybut `nazwa` przechowuje nazwę zestawu nadaną przez użytkownika. Umożliwia ona rozróżnienie wielu konfiguracji zapisanych przez tego samego klienta.
3. Atrybut `id_klienta` jest kluczem obcym wskazującym na tabelę `Klienci`. Pozwala na przypisanie zestawu do konkretnego użytkownika systemu.
4. Atrybut `data_utworzenia` określa datę utworzenia zestawu komputerowego. Informacja ta może być wykorzystywana do celów porządkowych oraz analitycznych.

## Tabela GPU

Tabela GPU przechowuje szczegółowe informacje techniczne dotyczące kart graficznych dostępnych w systemie. Encja ta zawiera parametry istotne z punktu widzenia kompatybilności zestawu komputerowego, takie jak zapotrzebowanie energetyczne, wymiary fizyczne oraz liczba obsługiwanych interfejsów wyjściowych. Tabela GPU jest powiązana z tabelą Produkty i uzupełnia ją o dane specyficzne dla kart graficznych. Strukturę tabeli GPU przedstawiono w tabeli 5.6.

Tabela 5.6: Struktura tabeli GPU

| Nazwa pola      | Typ danych   | Opis                                  |
|-----------------|--------------|---------------------------------------|
| id_gpu          | integer      | Klucz główny tabeli                   |
| chipset         | varchar(6)   | Rodzina lub seria układu graficznego  |
| vram            | smallint     | Pojemność pamięci VRAM w gigabajtach  |
| gddr            | varchar(6)   | Typ zastosowanej pamięci graficznej   |
| tdp             | smallint     | Zapotrzebowanie energetyczne karty    |
| sloty_obudowy   | numeric(2,1) | Liczba slotów zajmowanych w obudowie  |
| ilosc_wiatrakow | smallint     | Liczba wentylatorów karty graficznej  |
| dlugosc         | smallint     | Długość karty graficznej              |
| dual_bios       | smallint     | Informacja o obsłudze trybu dual BIOS |
| ilosc_hdmi      | smallint     | Liczba wyjść HDMI                     |
| ilosc_dp        | smallint     | Liczba wyjść DisplayPort              |

Poniżej przedstawiono opis poszczególnych pól tabeli GPU:

1. Atrybut id\_gpu jest kluczem głównym tabeli GPU. Służy do jednoznacznej identyfikacji karty graficznej oraz stanowi podstawę powiązań z tabelą Produkty.
2. Atrybut chipset określa rodzinę lub serię układu graficznego. Informacja ta wykorzystywana jest podczas filtrowania oraz prezentacji produktów.
3. Atrybut vram przechowuje pojemność pamięci graficznej karty. Parametr ten ma istotne znaczenie przy ocenie wydajności zestawu komputerowego.
4. Atrybut gddr określa typ zastosowanej pamięci graficznej, na przykład GDDR6 lub GDDR6X.
5. Atrybut tdp przechowuje informację o zapotrzebowaniu energetycznym karty graficznej. Dane te wykorzystywane są podczas analizy kompatybilności z zasilaczem.
6. Atrybut sloty\_obudowy określa liczbę slotów zajmowanych przez kartę w obudowie komputera. Parametr ten jest istotny przy weryfikacji zgodności z obudową.
7. Atrybut ilosc\_wiatrakow przechowuje liczbę wentylatorów zastosowanych w układzie chłodzenia karty graficznej.

8. Atrybut `dlugosc` określa długość fizyczną karty graficznej. Wartość ta wykorzystywana jest w procesie sprawdzania kompatybilności z obudową.
9. Atrybut `dual_bios` informuje o obecności trybu podwójnego BIOS-u. Umożliwia to rozróżnienie kart oferujących dodatkowe funkcje konfiguracyjne.
10. Atrybut `ilosc_hdmi` określa liczbę dostępnych wyjść HDMI.
11. Atrybut `ilosc_dp` określa liczbę dostępnych wyjść DisplayPort.

## Tabela CPU

Tabela CPU przechowuje szczegółowe informacje techniczne dotyczące procesorów dostępnych w systemie. Encja ta zawiera parametry istotne z punktu widzenia wydajności oraz kompatybilności zestawu komputerowego, takie jak typ gniazda, liczba rdzeni i wątków, zapotrzebowanie energetyczne oraz obsługa dodatkowych funkcji. Tabela CPU jest powiązana z tabelą Produkty i uzupełnia ją o dane specyficzne dla procesorów. Strukturę tabeli CPU przedstawiono w tabeli 5.7.

Tabela 5.7: Struktura tabeli CPU

| Nazwa pola  | Typ danych | Opis                                           |
|-------------|------------|------------------------------------------------|
| id_cpu      | integer    | Klucz główny tabeli                            |
| socket      | varchar(8) | Typ gniazda procesora                          |
| rdzenie     | smallint   | Liczba rdzeni procesora                        |
| watki       | smallint   | Liczba wątków procesora                        |
| zegar       | smallint   | Częstotliwość taktowania bazowego              |
| tdp         | smallint   | Zapotrzebowanie energetyczne procesora         |
| ma_cooler   | smallint   | Informacja o dołączonym chłodzeniu             |
| podkrecanie | smallint   | Obsługa podkręcania                            |
| ma_integre  | smallint   | Informacja o zintegrowanym układzie graficznym |

Poniżej przedstawiono opis poszczególnych pól tabeli CPU:

1. Atrybut id\_cpu jest kluczem głównym tabeli CPU. Służy do jednoznacznej identyfikacji procesora oraz stanowi podstawę powiązań z tabelą Produkty.
2. Atrybut socket określa typ gniazda procesora. Parametr ten jest wykorzystywany podczas sprawdzania kompatybilności procesora z płytą główną oraz systemem chłodzenia.
3. Atrybut rdzenie przechowuje liczbę fizycznych rdzeni procesora, co ma bezpośredni wpływ na jego wydajność.
4. Atrybut watki określa liczbę obsługiwanych wątków. Informacja ta wykorzystywana jest do oceny możliwości procesora w zastosowaniach wielowątkowych.
5. Atrybut zegar przechowuje częstotliwość taktowania bazowego procesora. Parametr ten ma znaczenie przy porównywaniu wydajności jednostek obliczeniowych.
6. Atrybut tdp określa zapotrzebowanie energetyczne procesora. Dane te wykorzystywane są w procesie doboru odpowiedniego zasilacza oraz systemu chłodzenia.
7. Atrybut ma\_cooler informuje, czy do procesora dołączone jest fabryczne chłodzenie.
8. Atrybut podkrecanie określa możliwość podkręcania procesora. Informacja ta pozwala na rozróżnienie modeli oferujących dodatkowe możliwości konfiguracyjne.

9. Atrybut `ma_integre` informuje o obecności zintegrowanego układu graficznego w procesorze.

## Tabela MOBO

Tabela MOBO przechowuje szczegółowe informacje techniczne dotyczące płyt głównych dostępnych w systemie. Encja ta stanowi kluczowy element procesu weryfikacji kompatybilności, ponieważ łączy parametry procesora, pamięci RAM oraz pozostałych podzespołów zestawu komputerowego. Tabela MOBO jest powiązana z tabelą Produkty i uzupełnia ją o dane specyficzne dla płyt głównych. Strukturę tabeli MOBO przedstawiono w tabeli 5.8.

Tabela 5.8: Struktura tabeli MOBO

| Nazwa pola                    | Typ danych  | Opis                                     |
|-------------------------------|-------------|------------------------------------------|
| <code>id_mobo</code>          | integer     | Klucz główny tabeli                      |
| <code>socket</code>           | varchar(8)  | Typ gniazda procesora                    |
| <code>ddr</code>              | varchar(4)  | Obsługiwany standard pamięci RAM         |
| <code>format</code>           | varchar(10) | Format płyty głównej                     |
| <code>max_ram</code>          | smallint    | Maksymalna obsługiwana ilość pamięci RAM |
| <code>ilosc_slotow_ram</code> | smallint    | Liczba slotów pamięci RAM                |
| <code>ilosc_slotow_m2</code>  | smallint    | Liczba slotów M.2                        |
| <code>ilosc_pcie_x16</code>   | smallint    | Liczba złączy PCIe x16                   |
| <code>ilosc_pcie_x1</code>    | smallint    | Liczba złączy PCIe x1                    |
| <code>ilosc_usb_3_0</code>    | smallint    | Liczba portów USB 3.0                    |
| <code>wifi</code>             | smallint    | Informacja o obsłudze Wi-Fi              |
| <code>ilosc_usb_2_0</code>    | smallint    | Liczba portów USB 2.0                    |
| <code>ilosc_usb_c</code>      | smallint    | Liczba portów USB typu C                 |
| <code>bluetooth</code>        | smallint    | Informacja o obsłudze Bluetooth          |
| <code>bios_typ</code>         | varchar(10) | Typ BIOS-u                               |
| <code>podkrecanie</code>      | smallint    | Obsługa podkreśniania                    |

Poniżej przedstawiono opis poszczególnych pól tabeli MOBO:

1. Atrybut `id_mobo` jest kluczem głównym tabeli MOBO. Służy do jednoznacznej identyfikacji płyty głównej oraz stanowi podstawę powiązań z tabelą Produkty.
2. Atrybut `socket` określa typ gniazda procesora. Parametr ten wykorzystywany jest podczas sprawdzania kompatybilności płyty głównej z procesorem.
3. Atrybut `ddr` przechowuje informację o obsługiwanym standardzie pamięci RAM, co umożliwia weryfikację zgodności z wybranymi modułami pamięci.
4. Atrybut `format` określa format płyty głównej, na przykład ATX lub mATX. Informacja ta jest wykorzystywana przy sprawdzaniu kompatybilności z obudową.

5. Atrybut `max_ram` określa maksymalną ilość pamięci RAM obsługiwaną przez płytę główną.
6. Atrybut `ilosc_slotow_ram` przechowuje liczbę slotów pamięci RAM dostępnych na płycie głównej.
7. Atrybut `ilosc_slotow_m2` określa liczbę dostępnych złączy M.2.
8. Atrybut `ilosc_pcie_x16` przechowuje liczbę złączy PCIe x16, wykorzystywanych głównie do podłączania kart graficznych.
9. Atrybut `ilosc_pcie_x1` określa liczbę złączy PCIe x1 przeznaczonych dla dodatkowych kart rozszerzeń.
10. Atrybut `ilosc_usb_3_0` przechowuje liczbę portów USB 3.0 dostępnych na płycie głównej.
11. Atrybut `wifi` informuje o obecności modułu Wi-Fi na płycie głównej.
12. Atrybut `ilosc_usb_2_0` określa liczbę portów USB 2.0.
13. Atrybut `ilosc_usb_c` przechowuje liczbę portów USB typu C.
14. Atrybut `bluetooth` informuje o obsłudze technologii Bluetooth.
15. Atrybut `bios_typ` określa typ zastosowanego BIOS-u.
16. Atrybut `podkrecanie` informuje o możliwości podkręcania podzespołów przy użyciu danej płyty głównej.

## Tabela RAM

Tabela RAM przechowuje szczegółowe informacje techniczne dotyczące pamięci operacyjnej wykorzystywanej w zestawach komputerowych. Encja ta zawiera parametry istotne z punktu widzenia kompatybilności oraz wydajności systemu, takie jak pojemność, taktowanie oraz standard pamięci. Tabela RAM jest powiązana z tabelą Produkty i uzupełnia ją o dane specyficzne dla modułów pamięci operacyjnej. Strukturę tabeli RAM przedstawiono w tabeli 5.9.



Tabela 5.9: Struktura tabeli RAM

| Nazwa pola       | Typ danych | Opis                             |
|------------------|------------|----------------------------------|
| id_ram           | integer    | Klucz główny tabeli              |
| pojemnosc_total  | smallint   | Łączna pojemność pamięci RAM     |
| liczba_modulow   | smallint   | Liczba modułów pamięci           |
| pojemnosc_modulu | smallint   | Pojemność pojedynczego modułu    |
| taktowanie       | smallint   | Częstotliwość taktowania pamięci |
| clock_latency    | smallint   | Opóźnienie pamięci (CL)          |
| ddr              | varchar(4) | Standard pamięci RAM             |
| profil           | char(1)    | Informacja o profilu XMP/EXPO    |

Poniżej przedstawiono opis poszczególnych pól tabeli RAM:

1. Atrybut `id_ram` jest kluczem głównym tabeli RAM. Służy do jednoznacznej identyfikacji modułu pamięci operacyjnej oraz stanowi podstawę powiązań z tabelą `Produkty`.
2. Atrybut `pojemnosc_total` określa całkowitą pojemność pamięci RAM dostępnej w zestawie.
3. Atrybut `liczba_modulow` przechowuje informację o liczbie modułów pamięci wchodzących w skład zestawu RAM.
4. Atrybut `pojemnosc_modulu` określa pojemność pojedynczego modułu pamięci operacyjnej.
5. Atrybut `taktowanie` przechowuje częstotliwość taktowania pamięci RAM. Parametr ten ma istotne znaczenie dla wydajności systemu.
6. Atrybut `clock_latency` określa opóźnienie pamięci RAM, oznaczane jako CL. Informacja ta wykorzystywana jest przy porównywaniu parametrów modułów.
7. Atrybut `ddr` określa standard pamięci RAM, na przykład DDR4 lub DDR5. Parametr ten wykorzystywany jest podczas sprawdzania kompatybilności z płytą główną.
8. Atrybut `profil` informuje o obsłudze profilu pamięci, takiego jak XMP lub EXPO, umożliwiającego pracę modułów z podwyższonymi parametrami.

## Tabela PSU

Tabela PSU przechowuje informacje techniczne dotyczące zasilaczy komputerowych dostępnych w systemie. Encja ta zawiera parametry istotne z punktu widzenia stabilności oraz bezpieczeństwa zestawu komputerowego, w szczególności moc zasilacza, jego format oraz certyfikację energetyczną. Tabela PSU jest powiązana z tabelą Produkty i uzupełnia ją o dane specyficzne dla zasilaczy. Strukturę tabeli PSU przedstawiono w tabeli 5.10.

Tabela 5.10: Struktura tabeli PSU

| Nazwa pola  | Typ danych  | Opis                                  |
|-------------|-------------|---------------------------------------|
| id_psu      | integer     | Klucz główny tabeli                   |
| moc         | smallint    | Moc znamionowa zasilacza              |
| format      | varchar(3)  | Format zasilacza                      |
| modularnosc | smallint    | Informacja o modularności okablowania |
| certyfikat  | varchar(10) | Certyfikat sprawności energetycznej   |

Poniżej przedstawiono opis poszczególnych pól tabeli PSU:

1. Atrybut `id_psu` jest kluczem głównym tabeli PSU. Służy do jednoznacznej identyfikacji zasilacza oraz stanowi podstawę powiązań z tabelą Produkty.
2. Atrybut `moc` określa moc znamionową zasilacza wyrażoną w watach. Parametr ten wykorzystywany jest podczas analizy zapotrzebowania energetycznego zestawu komputerowego.
3. Atrybut `format` określa standard obudowy zasilacza, na przykład ATX. Informacja ta ma znaczenie przy sprawdzaniu kompatybilności z obudową komputera.
4. Atrybut `modularnosc` informuje, czy zasilacz posiada modularne okablowanie. Parametr ten wpływa na estetykę oraz łatwość zarządzania przewodami w obudowie.
5. Atrybut `certyfikat` przechowuje informację o klasie sprawności energetycznej zasilacza, na przykład 80 PLUS Bronze, Gold lub Platinum.

## Tabela Cooler

Tabela COOLER przechowuje informacje techniczne dotyczące systemów chłodzenia procesora dostępnych w systemie. Encja ta zawiera parametry istotne z punktu widzenia kompatybilności mechanicznej oraz efektywności chłodzenia, takie jak wysokość chłodzenia, jego typ oraz liczba wentylatorów. Tabela COOLER jest powiązana z tabelą Produkty i uzupełnia ją o dane specyficzne dla systemów chłodzenia. Strukturę tabeli COOLER przedstawiono w tabeli 5.11.

Tabela 5.11: Struktura tabeli COOLER

| Nazwa pola         | Typ danych  | Opis                           |
|--------------------|-------------|--------------------------------|
| id_cooler          | integer     | Klucz główny tabeli            |
| wysokosc           | smallint    | Wysokość systemu chłodzenia    |
| typ                | varchar(16) | Typ chłodzenia                 |
| ilosc_wentylatorow | smallint    | Liczba wentylatorów            |
| rgb                | smallint    | Informacja o podświetleniu RGB |
| profil             | smallint    | Profil pracy chłodzenia        |

Poniżej przedstawiono opis poszczególnych pól tabeli COOLER:

1. Atrybut id\_cooler jest kluczem głównym tabeli COOLER. Służy do jednoznacznej identyfikacji systemu chłodzenia oraz stanowi podstawę powiązań z tabelą Produkty.
2. Atrybut wysokosc określa wysokość systemu chłodzenia. Parametr ten wykorzystywany jest podczas sprawdzania kompatybilności chłodzenia z obudową komputera.
3. Atrybut typ przechowuje informację o rodzaju systemu chłodzenia, na przykład powietrzne lub ciecżą.
4. Atrybut ilosc\_wentylatorow określa liczbę wentylatorów zastosowanych w systemie chłodzenia, co wpływa na jego wydajność.
5. Atrybut rgb informuje o obecności podświetlenia RGB w systemie chłodzenia.
6. Atrybut profil określa profil pracy systemu chłodzenia, umożliwiając dostosowanie jego działania do preferencji użytkownika.

## Tabela Dysk

Tabela DYSK przechowuje informacje techniczne dotyczące nośników danych wykorzystywanych w zestawach komputerowych. Encja ta zawiera parametry istotne z punktu widzenia wydajności oraz kompatybilności systemu, takie jak typ nośnika, zastosowany interfejs, format fizyczny oraz prędkości odczytu i zapisu. Tabela DYSK jest powiązana z tabelą Produkty i uzupełnia ją o dane specyficzne dla urządzeń pamięci masowej. Strukturę tabeli DYSK przedstawiono w tabeli 5.12.

Tabela 5.12: Struktura tabeli DYSK

| Nazwa pola       | Typ danych  | Opis                     |
|------------------|-------------|--------------------------|
| id_dysk          | integer     | Klucz główny tabeli      |
| typ              | varchar(5)  | Typ nośnika danych       |
| interfejs        | varchar(12) | Zastosowany interfejs    |
| format           | varchar(12) | Format fizyczny nośnika  |
| pojemnosc_gb     | smallint    | Pojemność nośnika danych |
| predkosc_zapisu  | smallint    | Prędkość zapisu danych   |
| predkosc_odczytu | smallint    | Prędkość odczytu danych  |

Poniżej przedstawiono opis poszczególnych pól tabeli DYSK:

1. Atrybut id\_dysk jest kluczem głównym tabeli DYSK. Służy do jednoznacznej identyfikacji nośnika danych oraz stanowi podstawę powiązań z tabelą Produkty.
2. Atrybut typ określa rodzaj nośnika danych, na przykład HDD lub SSD.
3. Atrybut interfejs przechowuje informację o zastosowanym interfejsie, takim jak SATA lub NVMe, co umożliwia sprawdzenie kompatybilności z płytą główną.
4. Atrybut format określa format fizyczny nośnika danych, na przykład 2.5 cala lub M.2.
5. Atrybut pojemnosc\_gb przechowuje pojemność nośnika danych wyrażoną w gigabajtach.
6. Atrybut predkosc\_zapisu określa maksymalną prędkość zapisu danych.
7. Atrybut predkosc\_odczytu określa maksymalną prędkość odczytu danych.

## 5.4 Komponenty i moduły systemu

System został zaprojektowany w sposób modułowy, co umożliwia jego czytelną strukturę oraz łatwą rozbudowę w przyszłości. Poszczególne moduły realizują konkretne zadania funkcjonalne systemu, a ich współdziałanie odbywa się po stronie backendu aplikacji.

### 5.4.1 Moduł kompatybilności podzespołów

Moduł kompatybilności podzespołów odpowiada za weryfikację technicznej zgodności komponentów wybieranych przez użytkownika podczas budowy zestawu komputerowego. Proces weryfikacji realizowany jest etapowo, zgodnie z określoną kolejnością reguł.

```
def check_cpu_mobo(cpu, motherboard):
    return cpu.socket == motherboard.socket
```

Pierwszym etapem weryfikacji jest sprawdzenie zgodności gniazda procesora z gniazdem płyty głównej. Jest to kluczowa reguła kompatybilności, bez której dalsza konfiguracja zestawu nie jest możliwa.

```
def check_ram_ddr(ram, motherboard):
    return ram.-ddr_type == motherboard.-ddr_type
```

Kolejnym krokiem jest sprawdzenie zgodności standardu pamięci RAM z płytą główną. Moduł analizuje typ pamięci DDR oraz obsługiwany standard przez płytę główną.

```
def check_power(cpu, gpu, psu):
    total_power = cpu.tdp + gpu.tdp
    return psu.power >= total_power
```

Dodatkowo weryfikowane jest zapotrzebowanie energetyczne zestawu w odniesieniu do mocy zasilacza. W przypadku wykrycia niezgodności system blokuje możliwość zapisania zestawu.

### 5.4.2 Moduł logiki rozmytej

Moduł logiki rozmytej został zastosowany w celu elastycznej klasyfikacji zestawów komputerowych do określonych kategorii użytkowych. W przeciwieństwie do klasycznych algorytmów decyzyjnych opartych na ostrych progach, zaproponowane rozwiązanie umożliwia ocenę konfiguracji w sposób niebinarny, z uwzględnieniem stopnia dopasowania zestawu do poszczególnych kategorii.

Podstawą działania modułu są funkcje przynależności zbiorów rozmytych, opisujące takie cechy zestawu jak łączna cena, wydajność oraz zapotrzebowanie energetyczne. Do opisu przynależności zastosowano trapezoidalną funkcję przynależności, która umożliwia płynne przejście pomiędzy stanami częściowej i pełnej przynależności.

```
def trapezoidal_membership(x, a, x1, x2, b):
    if x <= a or x >= b:
        return 0.0
```

```
elif a < x < x1:
    return (x - a) / (x1 - a)
elif x1 <= x <= x2:
    return 1.0
elif x2 < x < b:
    return (b - x) / (b - x2)
else:
    return 0.0
```

Na podstawie obliczonych stopni przynależności realizowany jest proces wnioskowania rozmytego typu Mamdaniego, w którym operator minimum pełni rolę koniunkcji logicznej. Wyniki poszczególnych reguł są następnie agregowane, a ostateczna klasyfikacja zestawu wyznaczana jest na podstawie kategorii o najwyższym stopniu dopasowania.

```
def classify_build_fuzzy(price, performance, power):
    mu_budget = trapezoidal_membership(price, 0, 1500, 3000, 5000)
    mu_performance = trapezoidal_membership(performance, 50, 70, 90, 100)
    mu_energy = trapezoidal_membership(power, 0, 200, 400, 700)

    score_budget = min(mu_budget, 1 - mu_energy)
    score_performance = min(mu_performance, mu_energy)
    score_energy = min(1 - mu_performance, mu_energy)

    scores = {
        "budzetowy": score_budget,
        "wydajny": score_performance,
        "energooszczędny": score_energy
    }

    return max(scores, key=scores.get), scores
```

Zastosowane podejście pozwala na uwzględnienie przypadków pośrednich, w których zestaw komputerowy nie spełnia jednoznacznie kryteriów jednej kategorii. Dzięki temu system generuje bardziej realistyczne i użyteczne rekomendacje, zwiększając jakość wsparcia decyzyjnego dla użytkownika końcowego.

Zależność pomiędzy wartością analizowanej cechy a stopniem jej przynależności do zbioru rozmytego opisano następującą zależnością:

$$\mu(x) = \begin{cases} 0 & x \leq a \\ \frac{x-a}{x_1-a} & a < x < x_1 \\ 1 & x_1 \leq x \leq x_2 \\ \frac{b-x}{b-x_2} & x_2 < x < b \\ 0 & x \geq b \end{cases}$$

### 5.4.3 Moduł zarządzania koszykiem i zamówieniami

Moduł zarządzania koszykiem odpowiada za obsługę procesów związanych z gromadzeniem produktów wybranych przez użytkownika oraz przygotowaniem ich do realizacji zamówienia.

```
@router.post("/cart/add")
def add_to_cart(product_id: int, quantity: int = 1):
    item = get_cart_item(product_id)
    if item:
        item.quantity += quantity
    else:
        create_cart_item(product_id, quantity)
```

System umożliwia dodawanie produktów do koszyka oraz automatyczne zwiększanie ilości w przypadku ponownego dodania tego samego produktu.

```
@router.get("/cart")
def get_cart():
    return fetch_cart_items()
```

Zawartość koszyka może zostać pobrana w dowolnym momencie, co pozwala na bieżące wyświetlanie aktualnego stanu koszyka w interfejsie użytkownika.

```
@router.delete("/cart/clear")
def clear_cart():
    remove_all_items()
```

Moduł umożliwia również całkowite wyczyszczenie koszyka, na przykład po zakończeniu procesu zakupowego.

## 5.5 Najważniejsze algorytmy

Jednym z kluczowych algorytmów systemu jest algorytm sprawdzania kompatybilności podzespołów. Algorytm działa sekwencyjnie i przerywa działanie w momencie wykrycia pierwszej niezgodności.

```
def check_compatibility(components):  
    if not check_cpu_mobo(components["cpu"], components["mobo"]):  
        return False  
    if not check_ram_ddr(components["ram"], components["mobo"]):  
        return False  
    if not check_power(components["cpu"], components["gpu"], components["psu"]):  
        return False  
    return True
```

Takie podejście umożliwia szybkie wykrycie błędów konfiguracji oraz czytelne wskazanie przyczyny problemu użytkownikowi.

```
@router.post("/builder/save")  
def save_build(components: list):  
    if not check_compatibility(components):  
        raise HTTPException(status_code=400)  
    save_to_database(components)
```

Fragment kodu odpowiada za zapis zestawu komputerowego po wcześniejszej weryfikacji kompatybilności. Zastosowane rozwiązanie zapewnia, że do bazy danych trafiają wyłącznie poprawne technicznie konfiguracje.



# Rozdział 6

## Weryfikacja i walidacja

Celem niniejszego rozdziału jest weryfikacja poprawności działania zaprojektowanego systemu. Część ta szczegółowo opisuje scenariusze testowe oraz analizuje wyniki testów funkcjonalnych, potwierdzające spełnienie założonych wymagań projektowych.

### 6.1 Cel weryfikacji systemu

Celem weryfikacji systemu było sprawdzenie poprawności działania zaprojektowanej systemu oraz ocena stopnia spełnienia założonych wymagań funkcjonalnych. Proces testowania miał na celu potwierdzenie, że wszystkie kluczowe moduły systemu działają zgodnie z przyjętymi założeniami projektowymi.

W ramach weryfikacji sprawdzono poprawność realizacji podstawowych funkcjonalności aplikacji, w tym procesu tworzenia zestawów komputerowych, obsługi koszyka oraz składania zamówień. Szczególną uwagę poświęcono mechanizmowi weryfikacji kompatybilności podzespołów, którego zadaniem jest identyfikacja konfiguracji niezgodnych technicznie.

Dodatkowo weryfikacja obejmowała ocenę stabilności działania warstwy backendowej oraz poprawności komunikacji pomiędzy frontendem a backendem. Przeprowadzone testy miały potwierdzić, że system reaguje w sposób przewidywalny na różne scenariusze użycia oraz zapewnia spójne i poprawne wyniki działania.

### 6.2 Metody testowania

W procesie weryfikacji systemu zastosowano testy funkcjonalne oraz testy scenariuszowe. Testy funkcjonalne polegały na sprawdzeniu poprawności działania poszczególnych funkcji aplikacji w odpowiedzi na określone dane wejściowe wprowadzane przez użytkownika.

Testy scenariuszowe zostały przeprowadzone w oparciu o realistyczne przypadki użycia systemu, obejmujące zarówno poprawne, jak i błędne konfiguracje zestawów komputerowych. Scenariusze testowe odzwierciedlały rzeczywiste działania użytkownika, takie jak dobór podzespołów, modyfikacja zestawu oraz próba utworzenia konfiguracji niezgodnej technicznie.

Testowanie realizowane było w sposób manualny poprzez interfejs użytkownika aplikacji. Takie podejście pozwoliło na jednoczesną weryfikację poprawności logiki biznesowej systemu oraz oceny działania warstwy frontendowej i backendowej podczas typowego użytkowania aplikacji.

## 6.3 Organizacja eksperymentów

Eksperymenty testowe zostały przeprowadzone w kontrolowanym środowisku testowym, które umożliwiała weryfikację poprawności działania wszystkich kluczowych funkcjonalności systemu. Testy obejmowały zarówno warstwę frontendową, jak i backendową aplikacji oraz ich współpracę z bazą danych.

Proces testowania systemu został zorganizowany w sposób uporządkowany i powtarzalny, a poszczególne etapy eksperymentów testowych przedstawiono poniżej:

1. Przygotowanie środowiska testowego obejmujące uruchomienie backendu aplikacji oraz bazy danych zawierającej przykładowe dane produktów i użytkowników.
2. Weryfikacja poprawności procesów logowania i rejestracji użytkowników oraz pobierania danych o dostępnych produktach.
3. Przeprowadzenie testów funkcjonalnych polegających na dodawaniu komponentów do zestawu, modyfikacji konfiguracji oraz analizie reakcji systemu na zmiany dokonywane przez użytkownika.
4. Sprawdzenie działania algorytmu weryfikacji kompatybilności oraz logiki rozmytej poprzez tworzenie zarówno poprawnych, jak i niepoprawnych konfiguracji zestawów komputerowych.
5. Testowanie procesu realizacji zamówienia, obejmujące przejście przez koszyk oraz zapis danych zamówienia w bazie systemu.
6. Rejestracja wyników testów oraz porównanie uzyskanych rezultatów z oczekiwanym zachowaniem systemu.

## 6.4 Przypadki testowe

W niniejszym podrozdziale przedstawiono wybrane przypadki testowe wykorzystane podczas weryfikacji poprawności działania systemu. Przypadki testowe obejmują zarówno konfiguracje poprawne, jak i niepoprawne, w celu sprawdzenia reakcji systemu na różne scenariusze użytkowania.

Testy skoncentrowane były głównie na mechanizmie weryfikacji kompatybilności podzespołów komputerowych, poprawności obsługi procesów użytkownika oraz realizacji podstawowych funkcjonalności aplikacji.

Tabela 6.1: Przykładowe przypadki testowe systemu

| ID | Test                       | Dane wejściowe                 | Oczekiwany wynik                | Wynik     |
|----|----------------------------|--------------------------------|---------------------------------|-----------|
| 1  | Zestaw kompatybilny        | CPU AM5 + MOBO AM5 + RAM DDR5  | Konfiguracja poprawna           | Pozytywny |
| 2  | Niezgodny socket CPU–MOBO  | CPU AM5 + MOBO LGA1700         | Odrzucenie konfiguracji         | Negatywny |
| 3  | Niezgodny standard RAM     | RAM DDR5 + MOBO DDR4           | Odrzucenie konfiguracji         | Negatywny |
| 4  | Zbyt długa karta graficzna | GPU > maks. długość obudowy    | Odrzucenie konfiguracji         | Negatywny |
| 5  | Zbyt słaby zasilacz        | PSU < zapotrzebowanie CPU/GPU  | Odrzucenie konfiguracji         | Negatywny |
| 6  | Zgodne chłodzenie CPU      | CPU + COOLER zgodny z socketem | Konfiguracja poprawna           | Pozytywny |
| 7  | Poprawna rejestracja       | Poprawne dane użytkownika      | Rejestracja zakończona sukcesem | Pozytywny |
| 8  | Błędne logowanie           | Niepoprawne hasło              | Odmowa dostępu                  | Negatywny |

## 6.5 Wyniki testów

Tabela 6.2: Przykładowe wyniki testów kompatybilności zestawów komputerowych.

| ID | Test                | Dane wejściowe                      | Problem                   | Wynik           |
|----|---------------------|-------------------------------------|---------------------------|-----------------|
| 1  | CPU + płyta główna  | Ryzen 5 7600 + B650                 | Brak                      | Kompatybilne    |
| 2  | CPU + płyta główna  | i5-12600K + B550                    | Różne sockety             | Niekompatybilne |
| 3  | RAM + płyta główna  | DDR5 6000 + B460                    | RAM DDR5 + MOBO DDR4      | Niekompatybilne |
| 4  | GPU + obudowa       | RTX 4070 (330 mm) + obudowa 300 mm  | Zbyt długie GPU           | Niekompatybilne |
| 5  | Zasilacz + CPU/GPU  | PSU 400W + RTX 3080                 | Zbyt mała moc             | Niekompatybilne |
| 6  | Chłodzenie + socket | Cooler AM4 + CPU LGA1700            | Niezgodne typy chłodzenia | Niekompatybilne |
| 7  | Zestaw kompletny    | CPU + GPU + RAM + Mobo + PSU zgodne | Brak                      | Kompatybilne    |
| 8  | Zestaw błędny       | 3 niezgodności (DDR, TDP, GPU)      | Złożona                   | Niekompatybilne |

## 6.6 Wykryte i usunięte błędy

W trakcie realizacji projektu oraz procesu testowania systemu wykryto kilka nieprawidłowości związanych głównie z obsługą danych oraz komunikacją pomiędzy poszczególnymi warstwami aplikacji. Błędy te miały charakter implementacyjny i zostały usunięte na etapie prac rozwojowych.

Jednym z napotkanych problemów były nieprawidłowości w przeliczaniu wartości cenowych produktów, wynikające z obsługi typu numeric po stronie bazy danych oraz jego mapowania w warstwie backendowej. W niektórych przypadkach wartości cenowe były błędnie interpretowane lub zaokrąglane. Problem został rozwiązany poprzez doprecyzowanie typów danych w modelach oraz ujednolicenie sposobu przetwarzania wartości liczbowych w aplikacji.

Dodatkowo wykryto błędy związane z niejednoznacznym mapowaniem typów danych w bibliotece ORM, co prowadziło do niepoprawnej walidacji wybranych parametrów technicznych podzespółów. Po analizie problemu wprowadzono korekty w definicjach modeli danych oraz regułach walidacyjnych.

W początkowej fazie testów pojawiły się również problemy konfiguracyjne związane z komunikacją pomiędzy frontendem a backendem, w tym błędy wynikające z nieprawidłowych ustawień środowiskowych. Po poprawieniu konfiguracji aplikacji oraz ujednoliceniu parametrów połączeń problemy te zostały całkowicie wyeliminowane.

Usunięcie wskazanych błędów pozwoliło na uzyskanie stabilnego działania systemu oraz poprawne funkcjonowanie mechanizmów walidacji i kompatybilności.

## 6.7 Ocena poprawności działania systemu

Na podstawie przeprowadzonych testów oraz uzyskanych wyników można stwierdzić, że zaprojektowany system działa poprawnie i spełnia założone wymagania funkcjonalne. Kluczowe mechanizmy aplikacji, w szczególności proces tworzenia zestawów komputerowych oraz weryfikacji kompatybilności podzespołów, funkcjonują zgodnie z przyjętymi założeniami projektowymi.

System prawidłowo identyfikuje konfiguracje niezgodne technicznie oraz poprawnie obsługuje przypadki poprawnych zestawów, umożliwiając ich dalszą realizację w procesie zakupowym. Przeprowadzone testy potwierdziły również stabilność działania warstwy backendowej oraz poprawność komunikacji pomiędzy frontendem a backendem.

Uzyskane wyniki testów potwierdzają, że działanie systemu jest zgodne z wymaganiami funkcjonalnymi oraz нефunkcjonalnymi określonymi w rozdziale trzecim pracy. Przeprowadzone przypadki testowe wykazały poprawną realizację kluczowych funkcji systemu, takich jak rejestracja i logowanie użytkowników, przeglądanie katalogu produktów, tworzenie zestawów komputerowych oraz automatyczna weryfikacja kompatybilności podzespołów.

System spełnił również założenia нефunkcjonalne w zakresie stabilności, czytelności komunikatów oraz przewidywalności działania. W szczególności mechanizm weryfikacji kompatybilności reagował w sposób kontrolowany i jednoznaczny na konfiguracje niezgodne technicznie, co potwierdza poprawność przyjętych założeń projektowych.



# Rozdział 7

## Podsumowanie i wnioski

Celem niniejszego rozdziału jest podsumowanie rezultatów uzyskanych podczas realizacji pracy inżynierskiej oraz ocena stopnia spełnienia założonych celów. Przedstawiono również trudności napotkane w trakcie realizacji projektu oraz możliwe kierunki dalszego rozwoju systemu.

### 7.1 Realizacja celów pracy

Głównym celem pracy było zaprojektowanie oraz implementacja systemu sklepu komputerowego z kreatorem zestawów PC, umożliwiającego użytkownikowi dobór kompatybilnych podzespołów komputerowych. Cel ten został osiągnięty poprzez opracowanie kompletnej architektury aplikacji obejmującej warstwę frontendową, backendową oraz relacyjną bazę danych.

W ramach realizacji projektu zaprojektowano znormalizowany model danych, zaimplementowano backend odpowiedzialny za logikę biznesową oraz mechanizmy weryfikacji kompatybilności podzespołów, a także przygotowano interfejs użytkownika umożliwiający interaktywne tworzenie zestawów komputerowych. Zrealizowane funkcjonalności odpowiadają na problem przedstawiony w rozdziale pierwszym, polegający na braku narzędzi umożliwiających automatyczną analizę zgodności komponentów w klasycznych sklepach internetowych.

### 7.2 Ocena otrzymanych rezultatów

Otrzymane rezultaty należy ocenić pozytywnie. Zaimplementowany system działa zgodnie z przyjętymi założeniami projektowymi i umożliwia poprawne tworzenie zestawów komputerowych wraz z ich weryfikacją pod kątem kompatybilności. Najważniejsze funkcje systemu, takie jak dobór podzespołów, analiza zgodności technicznej oraz obsługa koszyka i zamówień, zostały przetestowane i potwierdziły swoją poprawność.

Do mocnych stron rozwiązania należy zaliczyć przejrzystą architekturę, czytelny interfejs użytkownika oraz elastyczny model danych umożliwiający dalszą rozbudowę systemu. Szczególną wartość stanowi mechanizm sprawdzania kompatybilności, który pozwala na eliminację błędnych konfiguracji już na etapie tworzenia zestawu. System może być z powodzeniem wykorzystywany jako narzędzie wspomagające proces wyboru komponentów komputerowych.

## 7.3 Trudności napotkane podczas realizacji

Podczas realizacji projektu napotkano szereg trudności natury technicznej. Jednym z głównych wyzwań było zaprojektowanie uniwersalnego modelu danych, który umożliwiłby przechowywanie parametrów technicznych różnych typów podzespołów komputerowych, charakteryzujących się odmiennymi właściwościami.

Trudności pojawiły się również na etapie implementacji logiki weryfikacji kompatybilności, wymagającej uwzględnienia wielu zależności pomiędzy komponentami, takich jak zgodność gniazd procesora, standardów pamięci RAM czy ograniczeń wynikających z obudowy. Dodatkowym wyzwaniem była integracja warstwy frontendowej z backendem oraz zapewnienie poprawnej komunikacji pomiędzy poszczególnymi modułami systemu.

## 7.4 Możliwe kierunki dalszego rozwoju

Zaprojektowany system może stanowić podstawę do dalszego rozwoju. Jednym z możliwych kierunków jest rozbudowa mechanizmów logiki rozmytej poprzez zwiększenie liczby kategorii zestawów oraz bardziej precyzyjne reguły klasyfikacji. Możliwe jest również wprowadzenie automatycznego doboru podzespołów na podstawie zadanego budżetu użytkownika.

Dalszy rozwój systemu mógłby obejmować integrację z zewnętrznymi interfejsami API sklepów internetowych, wprowadzenie panelu administracyjnego do zarządzania katalogiem produktów, a także przygotowanie wersji mobilnej aplikacji. Potencjalnym kierunkiem rozwoju jest również zastosowanie algorytmów uczenia maszynowego w celu generowania bardziej zaawansowanych rekomendacji.

## 7.5 Wnioski końcowe

Zrealizowany projekt potwierdził możliwość stworzenia systemu, który w sposób zautomatyzowany wspiera użytkownika w procesie doboru kompatybilnych podzespołów komputerowych. Opracowane rozwiązanie łączy funkcjonalności typowe dla systemów e-commerce z mechanizmami analizy technicznej zestawów, co stanowi jego istotną wartość użytkową.



Podsumowując, praca spełniła założone cele projektowe i potwierdziła zdobyte w trakcie studiów umiejętności z zakresu projektowania baz danych, tworzenia aplikacji webowych oraz analizy problemów inżynierskich. Zaproponowany system jest rozwiązaniem kompletnym, spójnym i praktycznie użytecznym, mogącym stanowić podstawę do dalszego rozwoju i wdrożeń.



# Bibliografia

- [1] *Migrate PostgreSQL to Azure Database for PostgreSQL using Azure Database Migration Service*. 2025. URL: <https://learn.microsoft.com/en-us/azure/dms/tutorial-postgresql-azure-postgresql-online-portal> (term. wiz. 12.10.2025).
- [2] *Morele.net - sklep komputerowy*. 2025. URL: <https://www.morele.net/> (term. wiz. 17.11.2025).
- [3] *PC Part Dataset - Public Hardware Dataset for Computer Components*. 2025. URL: <https://github.com/docyx/pc-part-dataset> (term. wiz. 11.09.2025).
- [4] *X-kom - sklep komputerowy*. 2025. URL: <https://www.x-kom.pl/> (term. wiz. 17.11.2025).



# Dodatki



# Spis skrótów i symboli

- API interfejs programistyczny aplikacji (ang. *Application Programming Interface*)
- ASGI asynchroniczny interfejs serwera aplikacji (ang. *Asynchronous Server Gateway Interface*)
- ATX standard rozmiaru płyt głównych i obudów komputerowych (ang. *Advanced Technology eXtended*)
- CPU procesor centralny (ang. *Central Processing Unit*)
- CRUD podstawowe operacje na danych: tworzenie, odczyt, modyfikacja, usuwanie (ang. *Create, Read, Update, Delete*)
- CSS kaskadowe arkusze stylów (ang. *Cascading Style Sheets*)
- DDR standard pamięci RAM (*Double Data Rate*)
- ERD diagram związków encji (ang. *Entity–Relationship Diagram*)
- GPU procesor graficzny (ang. *Graphics Processing Unit*)
- HTML język znaczników stron WWW (ang. *HyperText Markup Language*)
- HTTP protokół komunikacji klient-serwer (ang. *HyperText Transfer Protocol*)
- JSON format danych tekstowych (ang. *JavaScript Object Notation*)
- ORM mapowanie obiektowo-relacyjne (ang. *Object–Relational Mapping*)
- PSU zasilacz komputerowy (ang. *Power Supply Unit*)
- RAM pamięć operacyjna (ang. *Random Access Memory*)
- REST styl architektury usług sieciowych (ang. *Representational State Transfer*)
- SQL język zapytań do baz danych (ang. *Structured Query Language*)
- SSD dysk półprzewodnikowy (ang. *Solid State Drive*)

TDP maksymalna ilość ciepła odprowadzana przez układ (ang. *Thermal Design Power*)

TS język programowania TypeScript (ang. *TypeScript*)

UI interfejs użytkownika (ang. *User Interface*)

UML zunifikowany język modelowania (ang. *Unified Modeling Language*)

URL adres zasobu internetowego (ang. *Uniform Resource Locator*)

WWW ogólnosiwiatowa sieć internetowa (ang. *World Wide Web*)



# Spis rysunków

|     |                                                           |    |
|-----|-----------------------------------------------------------|----|
| 4.1 | Widok strony głównej aplikacji . . . . .                  | 19 |
| 4.2 | Filtrowanie produktów z katalogu . . . . .                | 20 |
| 4.3 | Wybór produktów z przefiltrowanego katalogu . . . . .     | 20 |
| 4.4 | Widok koszyka użytkownika z dodanymi produktami . . . . . | 21 |
| 4.5 | Poprawne logowanie użytkownika . . . . .                  | 21 |
| 4.6 | Komunikat o niepoprawnych danych logowania . . . . .      | 22 |
| 4.7 | Widok kreatora zestawów komputerowych . . . . .           | 23 |
| 4.8 | Komunikat o niezgodności podzespołów . . . . .            | 24 |
| 5.1 | Schemat ideowy działania aplikacji . . . . .              | 28 |
| 5.2 | Diagram ERD bazy danych . . . . .                         | 36 |



# Spis tabel

|      |                                                                       |    |
|------|-----------------------------------------------------------------------|----|
| 5.1  | Struktura tabeli Produkty . . . . .                                   | 39 |
| 5.2  | Struktura tabeli Klienci . . . . .                                    | 40 |
| 5.3  | Struktura tabeli Koszyk . . . . .                                     | 41 |
| 5.4  | Struktura tabeli Zamówienia . . . . .                                 | 42 |
| 5.5  | Struktura tabeli Zestawy . . . . .                                    | 43 |
| 5.6  | Struktura tabeli GPU . . . . .                                        | 44 |
| 5.7  | Struktura tabeli CPU . . . . .                                        | 46 |
| 5.8  | Struktura tabeli MOBO . . . . .                                       | 47 |
| 5.9  | Struktura tabeli RAM . . . . .                                        | 49 |
| 5.10 | Struktura tabeli PSU . . . . .                                        | 50 |
| 5.11 | Struktura tabeli COOLER . . . . .                                     | 51 |
| 5.12 | Struktura tabeli DYSK . . . . .                                       | 52 |
| 6.1  | Przykładowe przypadki testowe systemu . . . . .                       | 59 |
| 6.2  | Przykładowe wyniki testów kompatybilności zestawów komputerowych. . . | 60 |